

**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN –
ĐHQG-HCM**
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN CUỐI KỲ

MÔN KIẾN TRÚC PHẦN MỀM

CRYPTO STRATEGY LAB

Nền tảng phân tích, kết hợp và đánh giá chiến lược giao dịch Crypto

Nhóm – Lớp:

Nhóm 09 – CSC13106 – 23KTPM01

Giảng viên:

PGS.TS. Trần Minh Triết

ThS. Trần Văn Quý

ThS. Nguyễn Huy Khánh

ThS. Ngô Ngọc Đăng Khoa

MSSV

Họ và tên

23127221

Nguyễn Tiến Luật

23127228

Phạm Văn Minh

23127281

Đặng Nghi Văn

23127494

Nguyễn Huỳnh Tiến

Thành phố Hồ Chí Minh, tháng 9 năm 2026

Tóm tắt

Crypto Strategy Lab là nền tảng phục vụ nghiên cứu và thử nghiệm chiến lược giao dịch crypto. Hệ thống tiếp nhận dữ liệu thị trường lịch sử và thời gian thực, cho phép người dùng cấu hình strategy đơn hoặc composite strategy, thực hiện backtest, đánh giá bằng các chỉ số định lượng, duy trì bảng xếp hạng Top-K và bổ sung tín hiệu sentiment từ tin tức. Trọng tâm của đề án không phải tìm ra chiến lược đầu tư sinh lời, mà là xây dựng một kiến trúc có thể thay đổi, mở rộng, phục hồi và kiểm chứng được theo yêu cầu môn học [1, 2].

Báo cáo trình bày kiến trúc theo mạch từ architectural drivers đến bằng chứng. Backend được tổ chức theo Modular Monolith với các capability module có public boundary rõ ràng; API và Worker được tách thành hai runtime; Sentiment Service chạy độc lập bằng Python/FastAPI. Các boundary biến thiên được bảo vệ bởi Market Data Provider Adapter, Strategy Plugin/Registry và Strategy Generator contract. Những tác vụ dài sử dụng Queue-Worker, Redis Streams, Transactional Outbox, at-least-once delivery và idempotency. PostgreSQL giữ nguồn dữ liệu nghiệp vụ chuẩn, trong khi Redis đảm nhiệm transport, cache và trạng thái tạm có thể phục hồi [9, 13, 12].

Phần đánh giá sử dụng quality attribute scenario và ATAM-lite để xem xét modifiability, replaceability, scalability, performance, realtime, reliability, observability và reproducibility. Báo cáo phân biệt rõ kết quả đã kiểm chứng, kết quả kiểm thử có điều khiển, phần còn thiếu bằng chứng LIVE và nội dung mới là kế hoạch. Các con số benchmark được trình bày cùng môi trường và giới hạn đo, không được diễn giải thành SLA production.

Từ khóa: Software Architecture, Modular Monolith, C4 Model, Strategy Plugin, Queue-Worker, Transactional Outbox, Redis Streams, Reproducibility, ATAM.

Mục lục

Tóm tắt	i
Danh mục hình	v
Danh mục bảng	vi
Thuật ngữ và chữ viết tắt	viii
1 Giới thiệu và bối cảnh bài toán	1
1.1 Bối cảnh	1
1.2 Vấn đề kiến trúc	1
1.3 Mục tiêu và stakeholder	2
1.4 Phạm vi và giới hạn	2
1.5 Đối chiếu yêu cầu chức năng	3
2 Architectural Drivers và Quality Attribute Scenarios	5
2.1 Từ yêu cầu chức năng đến ASR	5
2.2 Tám driver trọng yếu	5
2.3 Mẫu quality attribute scenario	6
2.4 Constraints và giả định	7
3 Kiến trúc tổng thể và các view	8
3.1 Cách tổ chức tài liệu kiến trúc	8
3.2 C4 Level 1 — System Context	8
3.3 C4 Level 2 — Container	9
3.4 C4 Level 3 — Backend Module View	10
3.5 Deployment View	11
3.6 Data ownership và mô hình dữ liệu khái niệm	12

4	Các quyết định kiến trúc và trade-off	14
4.1	Modular Monolith thay vì chia nhỏ sớm	14
4.2	API–Worker split	14
4.3	Ports, Adapter và Registry	14
4.4	Realtime bằng WebSocket	15
4.5	Queue–Worker và Transactional Outbox	15
4.6	Sentiment Service boundary	15
4.7	Frozen provenance và reproducibility	16
4.8	Ma trận quyết định và trade-off	16
5	Các luồng runtime trọng yếu	18
5.1	Historical Market Data	18
5.2	Realtime Candle và phục hồi	19
5.3	Strategy, Backtest và Evaluation	19
5.4	Search, Queue và Leaderboard	20
5.5	News và Sentiment	22
5.6	Failure isolation	23
5.7	Leaderboard provenance	24
6	Hiện thực hệ thống	25
6.1	Technology stack	25
6.2	Strategy Engine và thư viện strategy	26
6.3	Search Space và điều phối Experiment	26
6.4	Backtesting assumptions và metrics	26
6.5	Leaderboard và dữ liệu bất biến	27
6.6	Giao diện người dùng thực tế	27
7	Reliability, Scalability, Security và Observability	34
7.1	Backpressure và bounded work	34
7.2	Scale ngang Worker	34
7.3	Retry, recovery và deduplication	34
7.4	Observability	35
7.5	Security boundary	35
8	Đánh giá kiến trúc và Architecture Proof	37
8.1	Phương pháp ATAM-lite	37

8.2	Benchmark concurrency 1 và 3	38
8.2.1	Evidence scale F015: 100, 1.000 và 10.000 candidate	39
8.3	Tổng hợp evidence hiện tại	39
9	Cài đặt, build, kiểm thử và CI/CD	41
9.1	Môi trường local	41
9.2	Build và test strategy	41
9.3	CI/CD hiện có	42
10	Kết luận và hướng phát triển	43
10.1	Kết luận	43
10.2	Bài học kiến trúc	43
	Tài liệu tham khảo	44
A	Danh mục Architecture Decision Record	46
B	Requirement Traceability Matrix	47
C	ERD và Event Catalog	49
D	Bảng tự đánh giá và đóng góp thành viên	52
D.1	Nguyên tắc đọc bảng	52
D.2	Đóng góp thành viên	56
E	Liên kết bàn giao và hướng dẫn tái lập	57
E.1	Repository	57
E.2	Video demo và báo cáo nộp	57
E.3	Hướng dẫn setup, cài đặt và build	57
E.4	Tài liệu kiến trúc liên quan	57

Danh sách hình vẽ

3.1	C4 System Context của Crypto Strategy Lab	8
3.2	C4 Container View và các kênh giao tiếp	9
3.3	Dependency direction giữa các module backend	10
3.4	Deployment View cho môi trường local/demo	11
3.5	Mô hình dữ liệu khái niệm và quan hệ provenance	12
5.1	Luồng realtime, reconnect và gap recovery	19
5.2	Luồng Strategy–Backtest–Evaluation	19
5.3	Luồng tạo Experiment, xử lý Candidate và cập nhật Top-K	21
5.4	Luồng News collection và Sentiment Analysis	22
5.5	Các failure boundary và cơ chế phục hồi	23
5.6	Đường truy vết từ Leaderboard đến input bất biến	24
6.1	UI — Log In & Register	28
6.2	UI — Market Dashboard nhiều timeframe	28
6.3	UI — Strategy Composer	29
6.4	UI — Backtest Result và Trade detail	29
6.5	UI — Backtest detail	30
6.6	UI — Search results	30
6.7	UI — Search configuration	31
6.8	UI — Search process and leaderboard	32
6.9	UI — New Sentiment	33
C.1	ERD đầy đủ của PostgreSQL business schema	50

Danh sách bảng

1.1	Stakeholder và mối quan tâm chính	2
1.2	Phạm vi và non-goals của đề án	3
1.3	Tình trạng các capability chính	3
2.1	Architectural drivers của Crypto Strategy Lab	6
2.2	Các quality attribute scenario ưu tiên	7
3.1	Container catalog	9
3.2	Bounded Context và module trách nhiệm	10
3.3	Phân quyền sở hữu dữ liệu	13
4.1	Decision–driver–trade-off matrix	16
6.1	Technology stack từ source/configuration	25
6.2	Các metric đánh giá bắt buộc	27
7.1	Reliability tactics theo failure mode	35
8.1	Architecture proof theo scenario	37
8.2	Kết quả benchmark Backtest concurrency 1 so với 3	38
8.3	Tổng hợp evidence scale F015	39
8.4	Tổng hợp trạng thái kiểm chứng theo nhóm tiêu chí	40
9.1	Các quality gate chính	42
A.1	Danh mục ADR tại thời điểm lập báo cáo	46
B.1	Ánh xạ yêu cầu–kiến trúc–evidence	47
C.1	Event/message catalog rút gọn	50
D.1	Mức tự đánh giá của nhóm theo rubric	52

D.2 Thông tin đóng góp đã có trong bảng tự đánh giá 56

Thuật ngữ và chữ viết tắt

Ký hiệu	Tên đầy đủ	Ý nghĩa trong báo cáo
ADR	Architecture Decision Record	Bản ghi quyết định kiến trúc, phương án thay thế và hệ quả.
ASR	Architecturally Significant Requirement	Yêu cầu có ảnh hưởng trực tiếp đến cấu trúc hệ thống.
ATAM	Architecture Tradeoff Analysis Method	Phương pháp đánh giá kiến trúc bằng scenario và trade-off.
C4	Context, Container, Component, Code	Bộ view kiến trúc theo nhiều mức phóng đại.
CQRS	Command Query Responsibility Segregation	Tách mô hình ghi và đọc; dự án không triển khai CQRS đầy đủ.
DLQ	Dead Letter Queue	Nơi giữ message thất bại sau khi hết retry budget.
JWT	JSON Web Token	Access token dùng tại public API boundary.
MDD	Maximum Drawdown	Mức sụt giảm lớn nhất trong quá trình backtest.
MVP	Minimum Viable Product	Phạm vi sản phẩm tối thiểu cần chứng minh.
SLA	Service Level Agreement	Cam kết mức dịch vụ; các target demo trong báo cáo không phải SLA.
WSS	WebSocket Secure	Kênh WebSocket qua TLS cho môi trường triển khai.

Chương 1

Giới thiệu và bối cảnh bài toán

1.1 Bối cảnh

Thị trường cryptocurrency hoạt động liên tục và phát sinh khối lượng dữ liệu lớn theo nhiều cặp giao dịch, timeframe và nguồn thông tin. Một cây nến chuẩn biểu diễn giá mở cửa, cao nhất, thấp nhất, đóng cửa và khối lượng trong một khoảng thời gian. Trader thường phân tích chuỗi nến bằng Moving Average, RSI, Bollinger Bands, Support/Resistance và nhiều phương pháp khác. Tuy nhiên, một strategy đơn lẻ thường chỉ phù hợp với một số market regime nhất định: chiến lược bám xu hướng dễ nhiều khi thị trường đi ngang, trong khi chiến lược quá mua/quá bán có thể phát tín hiệu sớm trong một xu hướng mạnh.

Đề bài đặt câu hỏi rộng hơn việc cài đặt một indicator: liệu có thể tạo một nền tảng cho phép bổ sung strategy mới, kết hợp chúng, chạy backtest có kiểm soát, đánh giá và liên tục tìm candidate tốt hơn mà không phải viết lại toàn hệ thống hay không [1]. Vì vậy, đối tượng được thiết kế là một *experiment platform*, không phải một trading bot đặt lệnh thật.

1.2 Vấn đề kiến trúc

Một hiện thực ban đầu có thể gom việc gọi Binance, tính indicator, crawl tin tức, inference sentiment, backtest, xếp hạng, lưu database và gửi WebSocket vào một “God Service”. Cách làm đó có thể chạy được ở quy mô demo nhỏ nhưng tạo nhiều lý do thay đổi trong cùng một component. Khi yêu cầu thêm MACD, thay Binance bằng provider khác, đổi Random Search hoặc tăng số backtest, tác động sẽ lan sang các phần không liên quan.

Theo cách trình bày của seminar, báo cáo không bắt đầu bằng việc lựa chọn Kafka, Kubernetes hay microservices. Chuỗi lập luận được sử dụng là:

Business goal → ASR → Quality scenario → Decision/tactic → Technology → Evidence.

Mỗi công nghệ vì vậy phải trả lời được hai câu hỏi: nó giải quyết driver nào, và bằng chứng nào cho thấy quyết định có hiệu lực.

1.3 Mục tiêu và stakeholder

Các nhóm liên quan và mối quan tâm dùng để dẫn xuất architectural drivers được tổng hợp trong Bảng 1.1.

Bảng 1.1. Stakeholder và mối quan tâm chính

Stakeholder	Mục tiêu	Mối quan tâm kiến trúc
User/Trader	Theo dõi market, cấu hình strategy, chạy experiment và xem kết quả.	UI phản hồi, dữ liệu nhất quán, trạng thái rõ, kết quả truy vết được.
Nhóm phát triển	Thêm capability và sửa lỗi trong thời gian đồ án ngắn.	Boundary rõ, test tự động, thay đổi cục bộ, build có kiểm soát.
Team/Operator	Khởi động, demo và chẩn đoán hệ thống.	Health/readiness, log correlation, queue/job state, recovery.
Giảng viên/ Reviewer	Kiểm tra chất lượng kiến trúc thay vì chỉ xem tính năng.	C4 nhất quán với code, trade-off hợp lý, scenario và evidence tái kiểm tra được.
External providers	Cung cấp market data và news.	Rate limit, timeout, schema drift, disconnect và payload không tin cậy.

1.4 Phạm vi và giới hạn

Phạm vi MVP gồm dữ liệu lịch sử và realtime từ Binance qua adapter; tối đa bốn chart độc lập; bốn strategy cơ sở; composite strategy; backtest; bốn metric bắt buộc; Random Search với stop condition; Queue–Worker; Top-K Leaderboard; news collection; sentiment analysis; và provenance cho Experiment. Bảng 1.2 phân biệt rõ phần trong và ngoài phạm vi.

Bảng 1.2. Phạm vi và non-goals của đề án

Trong phạm vi	Ngoài phạm vi/không tuyên bố
Nghiên cứu và so sánh strategy trên dữ liệu lịch sử.	Giao dịch tiền thật, quản lý ví, custody hoặc order execution.
Realtime chart và multi-timeframe.	Cam kết dữ liệu provider luôn khả dụng hoặc tuyệt đối chính xác.
Plugin/Registry, composite và replaceable generator.	Upload và chạy plugin không tin cậy ở runtime.
Async jobs, retry, recovery và Top-K projection.	Exactly-once delivery hoặc zero-downtime production SLA.
News/Sentiment qua service boundary có version.	Khuyến nghị đầu tư hoặc cam kết lợi nhuận.
Docker cho Sentiment và local infrastructure.	Kubernetes, Kafka, service mesh hoặc microservice cho mọi module.

1.5 Đối chiếu yêu cầu chức năng

Repository hiện có source code và automated test cho các capability chính. Tuy nhiên, mức hiện thực và mức bằng chứng demo LIVE là hai khái niệm khác nhau. Bảng 1.3 dùng trạng thái implementation ở cấp repository; kết quả kiểm chứng chi tiết được trình bày tại Chương 8.

Bảng 1.3. Tình trạng các capability chính

Capability	Tình trạng code	Dấu vết trong repository
Market Data	Implemented	Binance REST/WebSocket adapter, canonical Candle, fixture provider, retry và reconnect.
Multi-timeframe UI	Implemented	Bốn panel giữ pair/timeframe và subscription state độc lập.
Strategy Engine	Implemented	Strategy contract, descriptor, parameter schema, Plugin/Registry.
Strategy Library	Implemented	MA Crossover, RSI, Bollinger Bands và Support/Resistance.
Composite Strategy	Implemented	Majority Vote, Weighted Vote và versioned composite definition.

Capability	Tình trạng code	Dấu vết trong repository
Search	Implemented	Random Search, deterministic seed/state, finite stop condition và bounded Search Space.
Backtest/ Evaluation	Implemented	Historical simulation, Trade/Result bất biến và bốn metric bắt buộc.
Leaderboard	Implemented	Top-K projection, deterministic tie-break và revision.
Reliable Worker	Implemented	Redis Streams, Consumer Group, Outbox, retry, reclaim, idempotency và DLQ.
News/Sentiment	Implemented	RSS normalization, deduplication, analysis lifecycle và FastAPI model runtime.
Public API/Web	Implemented	REST, JWT authorization, WebSocket ticket, reconciliation và các route chính.
Full LIVE demo	Implemented	Controlled gates có evidence; authenticated LIVE journey và video đã có.

Chương 2

Architectural Drivers và Quality Attribute Scenarios

2.1 Từ yêu cầu chức năng đến ASR

Không phải mọi yêu cầu đều ảnh hưởng kiến trúc ở cùng mức. Màu sắc của nút bấm thường là quyết định presentation cục bộ; ngược lại, yêu cầu “News Service lỗi nhưng Market Chart vẫn chạy” buộc hệ thống có failure boundary. Tương tự, yêu cầu tăng từ hàng trăm lên nhiều nghìn candidate tác động đến execution model, queue, persistence và observability. Đây là các ASR cần được ưu tiên trước khi chọn technology stack [3, 5].

2.2 Tám driver trọng yếu

Bảng 2.1 liên kết từng quality attribute với câu hỏi kiểm chứng và tác động trực tiếp lên cấu trúc hệ thống.

Bảng 2.1. Architectural drivers của Crypto Strategy Lab

Driver	Câu hỏi	Tác động cấu trúc
Modifiability	Thêm MACD phải sửa bao nhiêu nơi?	Strategy contract, Plugin/Registry, module boundary.
Replaceability	Đổi generator/provider/model có làm downstream đổi?	Port/adaptor, registry và versioned contract.
Scalability	Tăng candidate có thể tăng Worker không?	Queue, Consumer Group, bounded concurrency và DB plan.
Performance	Search dài có giữ HTTP request?	Async acceptance, batching, backpressure và projection.
Realtime	Candle mới đến bốn chart thế nào?	Multiplex WebSocket, coalescing, snapshot và reconciliation.
Reliability	Disconnect/crash được phục hồi ra sao?	Retry, reclaim, Outbox, idempotency và degraded state.
Observability	Có biết job nào lỗi và backlog bao nhiêu?	Correlation identity, progress, log và health endpoint.
Reproducibility	Top-K dùng dữ liệu và strategy version nào?	Frozen Manifest, checksum, fingerprint và immutable Result.

2.3 Mẫu quality attribute scenario

Nhóm sử dụng cấu trúc Source–Stimulus–Environment–Artifact–Response–Measure (S–S–E–A–R–M). Measure phải kiểm tra được và được hiểu là target của môi trường demo, không phải production SLA. Các scenario ưu tiên và measure tương ứng được trình bày tại Bảng 2.2.

Bảng 2.2. Các quality attribute scenario ưu tiên

ID	Attribute	Stimulus/Artifact	Response	Measure mục tiêu
QA-01	Modifiability	Thêm MACD vào Strategy capability.	Thêm plugin, descriptor và test qua Registry.	Không sửa Backtester, Evaluator, Leaderboard hoặc UI business logic.
QA-02	Replaceability	Thêm generator bên cạnh Random Search.	Resolve theo ID/version và trả Candidate Definition chuẩn.	Queue contract và downstream không đổi.
QA-03	Modifiability	Chọn Fixture/OKX thay Binance.	Adapter normalize payload/error sang canonical model.	Public API và frontend không đổi.
QA-04	Reliability	Binance WebSocket disconnect.	Báo reconnecting, backoff, backfill, sort và deduplicate.	Không thiếu/trùng closed Candle trong gap; target demo phục hồi $\leq 30s$.
QA-05	Scalability	Tăng Worker từ 1 lên 3.	Consumer Group phân phối job; reclaim và dedup.	Target $\geq 2 \times$ baseline, không duplicate Result.
QA-06	Fault isolation	Sentiment process timeout/down.	Retry giới hạn, circuit breaker và degraded state.	Market và technical Backtest tiếp tục; target báo degraded $\leq 5s$.
QA-07	Reproducibility	Reproduce một Top-K result.	Validate version/checksum và tạo reproduction run mới.	Trade sequence, metrics và fingerprint khớp; không overwrite bản gốc.
QA-08	Observability	Candidate queue/retry/fail/new best.	Persist progress và phát revision/status.	Trace được bằng correlationId, experimentId, candidateId và jobId.

2.4 Constraints và giả định

Nhóm có bốn thành viên và thời gian đề án giới hạn, do đó chi phí vận hành là constraint quan trọng. Binance, News Provider và Supabase là external dependencies không kiểm soát được availability. Hệ thống không đặt lệnh giao dịch và chỉ xử lý dữ liệu public cho mục đích nghiên cứu. Những scenario cần nhiều host, production traffic hoặc authenticated LIVE session được ghi nhận như khoảng trống kiểm chứng thay vì suy diễn từ unit test.

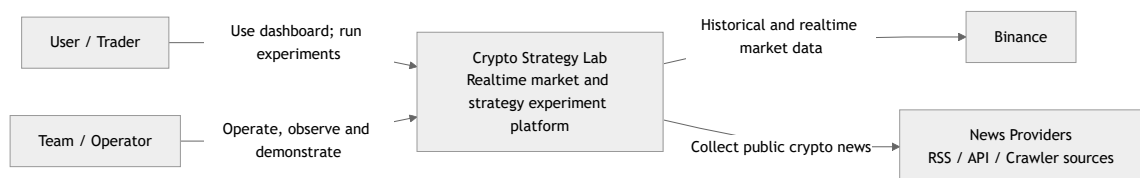
Chương 3

Kiến trúc tổng thể và các view

3.1 Cách tổ chức tài liệu kiến trúc

Một view kiến trúc phải trả lời một câu hỏi cụ thể. System Context cho biết hệ thống giao tiếp với ai; Container cho biết các runtime và data store; Component/Module cho biết trách nhiệm và dependency direction; Dynamic View mô tả tương tác theo thời gian. Cách tách view tránh việc nhồi framework, database và module vào cùng một hình [4, 6]. Việc nhóm khái niệm theo capability và giữ ngôn ngữ nhất quán giữa các boundary cũng dựa trên tư duy Domain-Driven Design [8].

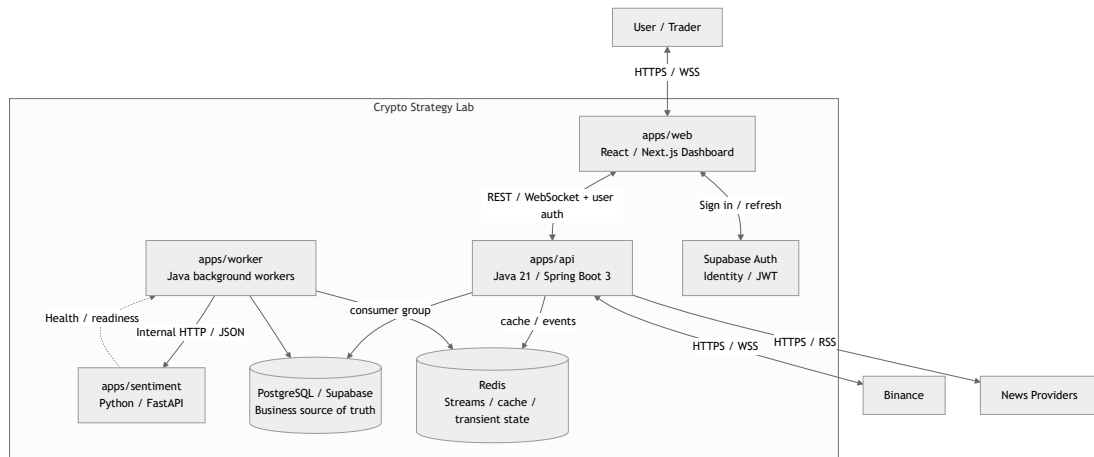
3.2 C4 Level 1 — System Context



Hình 3.1. C4 System Context của Crypto Strategy Lab

Boundary của hệ thống bao gồm Web Dashboard, public API/WebSocket, các capability nghiệp vụ, Worker, persistence và Sentiment runtime. Binance và News Providers nằm ngoài boundary; payload từ các nguồn này được xem là không tin cậy và phải qua adapter. User không truy cập trực tiếp Binance, business tables hoặc Sentiment Service.

3.3 C4 Level 2 — Container



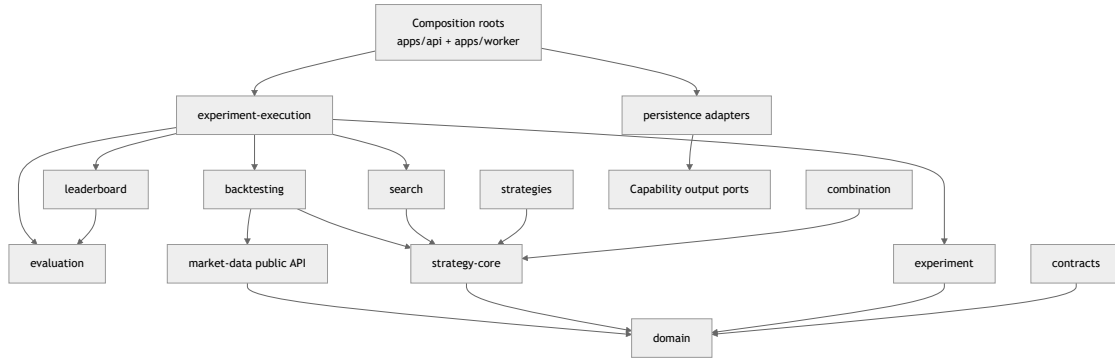
Hình 3.2. C4 Container View và các kênh giao tiếp

Vai trò, interface và hướng scale của từng container được đối chiếu trong Bảng 3.1.

Bảng 3.1. Container catalog

Container	Trách nhiệm	Interface	Scale strategy
apps/web	UI, chart, subscription state, progress và result view.	REST, WebSocket, Supabase Auth.	Web replica/CDN khi cần.
apps/api	Public contract, validation, WebSocket gateway và command orchestration.	HTTP/JSON, WebSocket, JDBC.	Scale sau khi kiểm chứng state strategy.
apps/worker	Search coordination, Backtest/Evaluation, Ranking và News jobs.	Redis Streams, JDBC, internal HTTP.	Tăng instance trong Consumer Group.
apps/sentiment	Stateless text inference có model provenance.	Internal HTTP/JSON, health/readiness.	Nhiều instance độc lập nếu inference là bottleneck.
PostgreSQL	Business source of truth và Outbox.	JDBC/PostgreSQL.	Pool/query/index plan là giới hạn chính.
Redis	Streams, pending delivery, cache và progress tạm.	Redis protocol.	Điều chỉnh topology theo measurement.

3.4 C4 Level 3 — Backend Module View



Hình 3.3. Dependency direction giữa các module backend

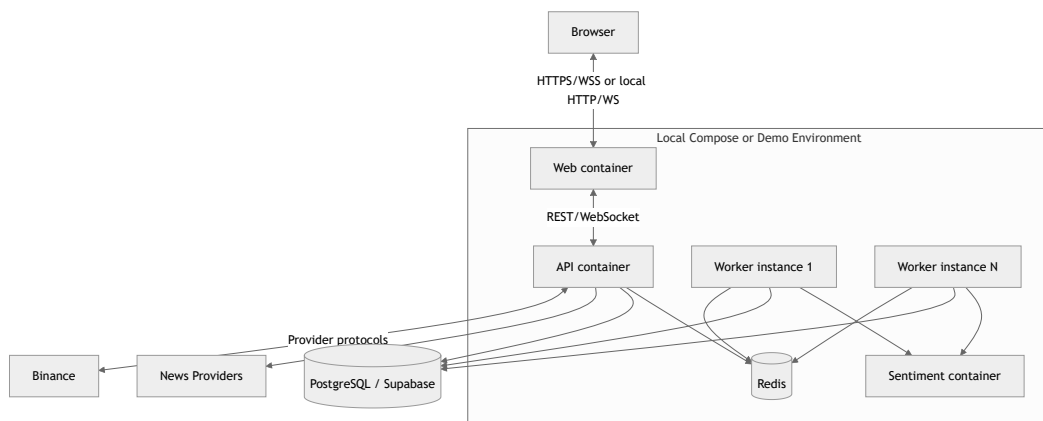
Backend Java có hai composition root là API và Worker. Business capability chỉ công khai kiểu và port ổn định trong package `api`; implementation nằm trong `internal`. Persistence triển khai output port của capability owner, không trở thành nơi chứa business policy. Các rule này được kiểm tra bằng Gradle module graph và ArchUnit, bao gồm quy tắc không import chéo implementation, không tạo cycle và giữ Strategy contract độc lập Spring/JDBC. Ranh giới trách nhiệm và các hành vi bị cấm ở mỗi module được tóm tắt tại Bảng 3.2.

Bảng 3.2. Bounded Context và module trách nhiệm

Context/module	Trách nhiệm	Không được làm
market-data	Canonical Candle, provider port, Binance/fixture adapter và recovery.	Để provider model thoát ra public contract.
strategy-core	Strategy, Signal, descriptor, schema và Registry.	Gọi network, database hoặc phụ thuộc Spring.
strategies	MA, RSI, Bollinger, Support/Resistance plugin.	Điều phối Search hoặc Backtest.
combination	Composite factory, Majority/Weighted policy.	Tính metrics hoặc Ranking.
experiment	Manifest, lifecycle, version và reproduction identity.	Chứa Strategy/Backtest algorithm.
search	Generator Registry, candidate proposal và stop condition.	Chạy Backtest hoặc cập nhật Top-K.

Context/module	Trách nhiệm	Không được làm
experiment-execution	Điều phối xuyên capability và durable decision.	Sở hữu thuật toán capability hoặc truy cập table trực tiếp.
backtesting	Mô phỏng execution, Trade và Result.	Biết generator hoặc Ranking implementation.
evaluation	Tính metric versioned từ Backtest Result.	Tạo Strategy hoặc Top-K.
leaderboard	Score, tie-break, Top-K và revision.	Chạy Search/Backtest.
news	Provider contract, normalize/dedup, sentiment ownership.	Phụ thuộc model Python cụ thể.
persistence	JDBC/Redis adapter cho output port.	Chứa business policy hoặc bị capability import ngược.

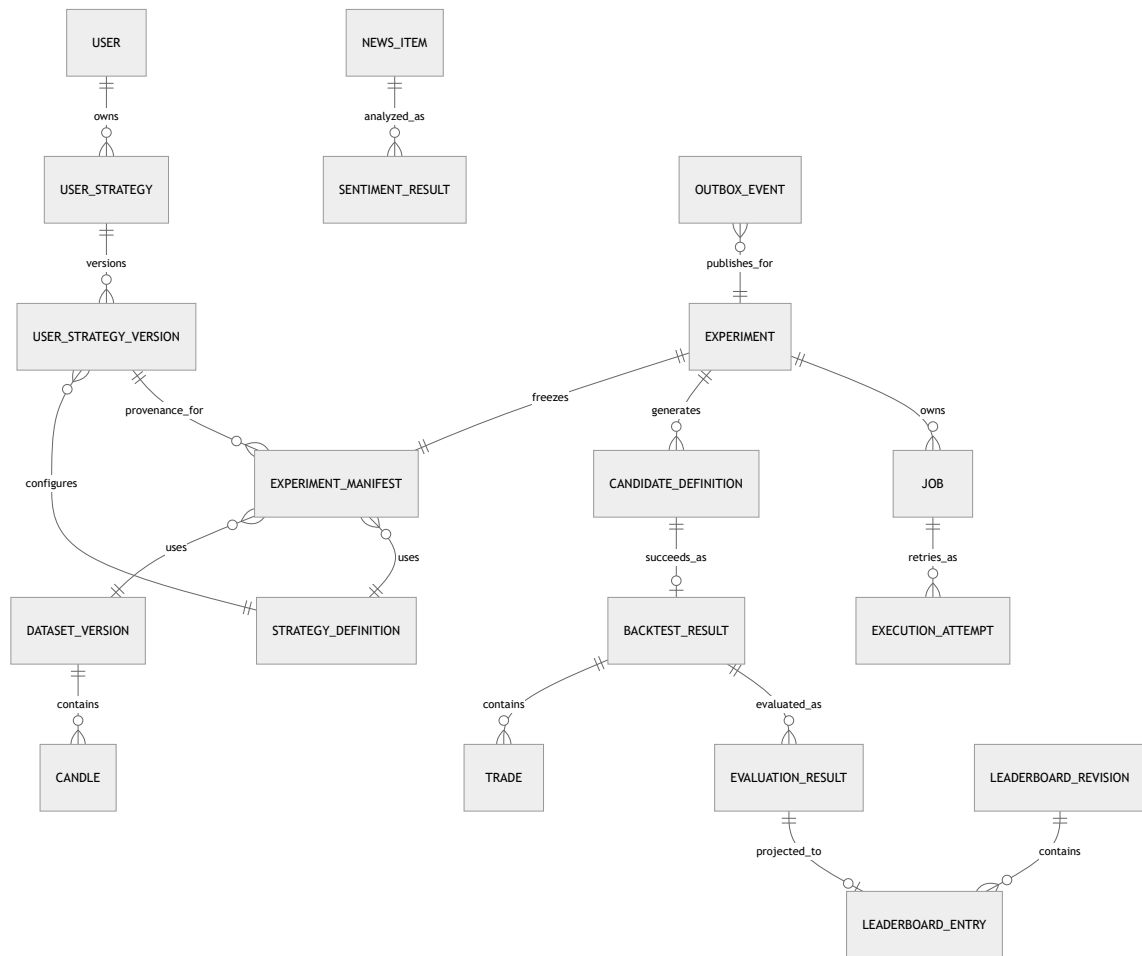
3.5 Deployment View



Hình 3.4. Deployment View cho môi trường local/demo

API và Worker là hai JVM process khác nhau dù dùng lại các capability module. Việc tách runtime giúp HTTP/WebSocket không chia cùng failure domain và thread pool với Backtest dài. Sentiment Service được container hóa riêng do dependency ML, startup time và failure mode khác Java. PostgreSQL/Supabase nằm ngoài local Compose trong cấu hình hiện tại; Redis và Sentiment có thể chạy local/container.

3.6 Data ownership và mô hình dữ liệu khái niệm



Hình 3.5. Mô hình dữ liệu khái niệm và quan hệ provenance

Bảng 3.3 chỉ rõ capability owner, nguồn dữ liệu chuẩn và invariant tương ứng; đây là cơ sở để tránh shared-database coupling không kiểm soát.

Bảng 3.3. Phân quyền sở hữu dữ liệu

Capability	Dữ liệu sở hữu	Nguồn chuẩn	Quy tắc
Market Data	Asset, pair, Candle, Dataset Version.	PostgreSQL	Closed Candle idempotent; Dataset có checksum.
Strategy	Strategy/User Strategy/Composite version.	PostgreSQL	Published version bất biến; descriptor có fingerprint.
Experiment	Manifest, Candidate, Job, Attempt.	PostgreSQL	Manifest đóng băng input; attempt tách khỏi logical job.
Backtest/ Evaluation	Result, Trade, metrics.	PostgreSQL	Result thành công bất biến và liên kết Candidate.
Leaderboard	Revision và Entry.	PostgreSQL	Projection có thể rebuild từ Evaluation Result.
News	News Item, analysis lifecycle, Sentiment Result.	PostgreSQL	Dedup bằng source/content identity; model version được lưu.
Platform	Outbox, processed message, idempotency record.	PostgreSQL	Redis mất không làm mất business truth.

Chương 4

Các quyết định kiến trúc và trade-off

4.1 Modular Monolith thay vì chia nhỏ sớm

Nhóm chọn Modular Monolith cho backend cốt lõi theo ADR-0001. Quyết định phù hợp với quy mô nhóm, thời gian đồ án và nhu cầu giữ transaction nội bộ đơn giản. Boundary được biểu diễn bằng Gradle subproject, public package và architecture test thay vì chỉ là quy ước. API/Worker có thể reuse capability mà không nhân bản domain model.

Giá phải trả là mọi module vẫn được build và release cùng repository; boundary có thể suy thoái nếu bypass public contract. Nhóm giảm rủi ro bằng dependency matrix, ArchUnit và ADR review. Microservices chỉ được dùng tại Sentiment boundary, nơi có driver thật về runtime/dependency/failure isolation.

4.2 API–Worker split

Public API nhận command, validate, ghi durable state và trả ID sớm. Worker thực thi Search, Backtest, Evaluation và background News/Sentiment. Boundary này ngăn tác vụ CPU dài giữ HTTP request, hỗ trợ retry và cho phép tăng Worker mà không sửa core contract. Đổi lại, người dùng quan sát eventual consistency: trạng thái ban đầu có thể là QUEUED, sau đó mới chuyển sang RUNNING/COMPLETED.

4.3 Ports, Adapter và Registry

Market Data Provider là output port; Binance và Fixture là adapter. Mọi payload được normalize sang canonical Candle trước khi đi vào domain. Nhờ đó, frontend và Strategy không phụ thuộc symbol, interval hay error shape riêng của Binance.

Strategy Plugin cung cấp descriptor, semantic version, parameter schema và hàm phân tích trả BUY/SELL/HOLD. Registry giữ danh sách trusted plugin compile-time. Việc thêm strategy mới tập trung trong module strategies và composition registration, thay vì thêm

chuỗi `if/else` tại Controller, Backtester và UI. Đây là ứng dụng Strategy/Registry/Dependency Injection theo mục tiêu extensibility [7].

Strategy Generator cũng được bảo vệ bằng contract. Random Search hiện thực interface chuẩn và sinh Candidate Definition; generator khác có thể được đăng ký theo ID/version. Search không chạy Backtest, còn Backtester không biết candidate được sinh bằng thuật toán nào.

4.4 Realtime bằng WebSocket

REST phù hợp cho snapshot, history và command; WebSocket phát update theo subscription. API multiplex subscription theo pair/timeframe, coalesce intermediate open-Candle update khi outbound chậm và không bỏ closed Candle hoặc completion event. Client giữ event identity/revision để bỏ stale update và gọi REST reconciliation sau reconnect. Trade-off là lifecycle, reconnect và ordering phức tạp hơn polling đơn giản.

4.5 Queue–Worker và Transactional Outbox

Một command tạo Experiment cần ghi Manifest, Job và ý định publish trong cùng database transaction. Nếu ghi database thành công nhưng publish Redis thất bại, Outbox Publisher có thể thử lại. Nếu publish thành công nhiều lần, consumer dùng stable message/job identity và durable processed record để ngăn business effect trùng. Mô hình này chấp nhận at-least-once delivery thay vì tuyên bố exactly-once [10, 11].

ACK chỉ được gửi sau durable commit. Worker crash trước ACK để message ở pending list; consumer khác có thể reclaim sau idle boundary. Lỗi transient dùng bounded retry/backoff; lỗi hết budget được đưa vào DLQ và trạng thái failure được lưu có cấu trúc. Redis cung cấp transport nhanh nhưng không giữ bản duy nhất của Manifest, Job, Result hoặc Leaderboard.

4.6 Sentiment Service boundary

Python/FastAPI được tách vì TensorFlow/Keras có dependency, lifecycle và resource profile khác Java. Service nhận text đã normalize, không tự crawl URL và không truy cập shared database. Contract trả label, confidence, polarity score và model/preprocessing version. Worker chịu timeout, retry, circuit breaker và persist lifecycle; failure của Sentiment không được lan sang Market hoặc technical Backtest.

4.7 Frozen provenance và reproducibility

Experiment Manifest đóng băng Dataset version/checksum, Strategy reference/version/parameters, Backtest assumptions, Search config, metric/ranking version, software version và git commit. Candidate, Result, Trade và Evaluation Result giữ quan hệ bất biến với Manifest. Reproduction tạo một run mới và so sánh fingerprint/trade/metrics; không overwrite kết quả gốc. Cơ chế này biến một dòng Leaderboard từ “con số hiện tại” thành một bằng chứng có nguồn gốc.

4.8 Ma trận quyết định và trade-off

Bảng 4.1 đặt mỗi quyết định cạnh driver, lợi ích và chi phí để công nghệ không bị trình bày như mục tiêu tự thân.

Bảng 4.1. Decision–driver–trade-off matrix

Quyết định	Driver	Lợi ích	Giá phải trả/rủi ro
Modular Monolith	Modifiability	Phát triển/deploy đơn giản, transaction nội bộ.	Cần test để boundary không suy thoái.
API/Worker split	Performance, isolation	HTTP phản hồi nhanh; scale compute riêng.	Eventual consistency, thêm process vận hành.
Plugin/Registry	Extensibility	Thêm Strategy cục bộ, catalog tự mô tả.	Quản lý schema, identity và version.
Provider Adapter	Replaceability	Core/frontend độc lập provider payload.	Mapping/error translation và contract test.
WebSocket	Realtime	Push update hiệu quả, subscription linh hoạt.	Reconnect, ordering, backpressure phức tạp.
Redis Streams	Scalability	Consumer Group, pending/reclaim, scale Worker.	Duplicate, queue operation và Redis availability.
Transactional Outbox	Reliability	Không mất ý định publish sau DB commit.	Publisher, cleanup và observability bổ sung.

Quyết định	Driver	Lợi ích	Giá phải trả/rủi ro
At-least-once + idempotency	Correctness	Retry/recovery an toàn hơn.	Stable identity và durable dedup state.
Sentiment service riêng	Isolation, replaceability	Tách ML runtime/ model lifecycle.	HTTP contract, timeout và container mới.
Frozen Manifest	Reproducibility	Truy vết và tái lập exact input.	Storage/version governance và validation.
Leaderboard projection	Read performance	Query Top-K nhanh, revision rõ.	Reconciliation/rebuild khi projection lệch.
Không full CQRS/Event Sourcing	Simplicity	Giảm độ phức tạp MVP.	Không có generic replay/temporal state toàn hệ thống.

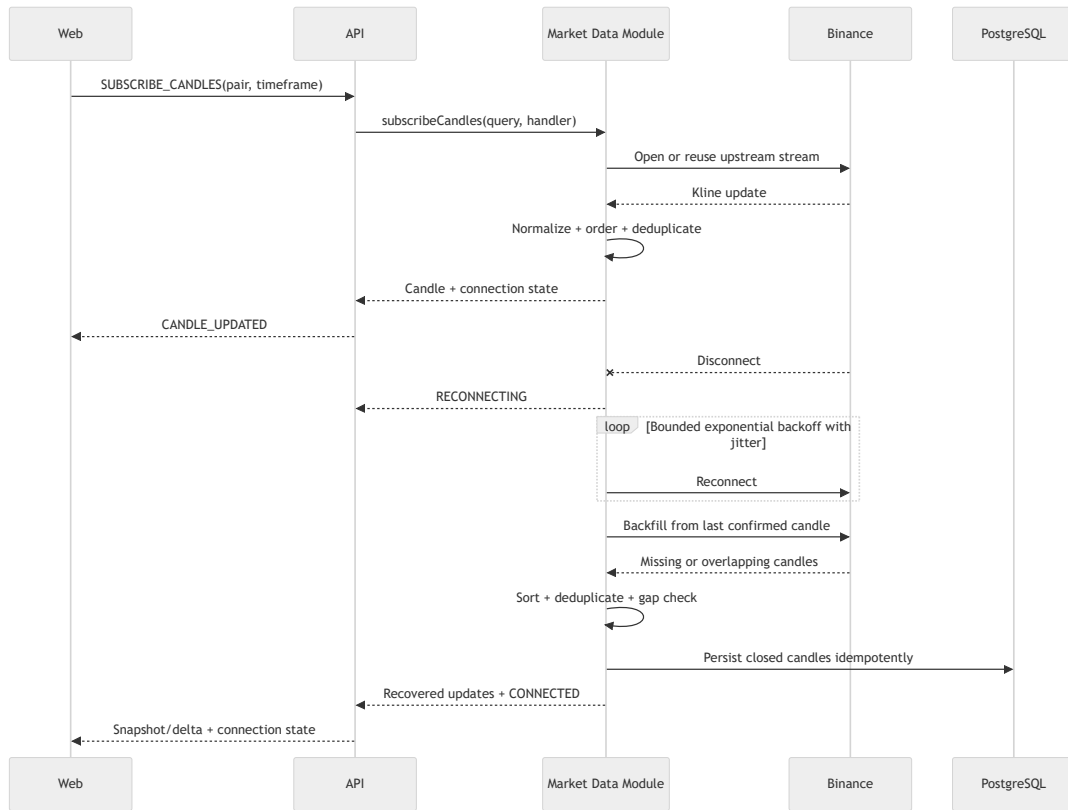
Chương 5

Các luồng runtime trọng yếu

5.1 Historical Market Data

Khi người dùng chọn pair, timeframe và range, Web gửi query chuẩn đến API. Market Data Module kiểm tra cache; cache miss dẫn đến các request phân trang qua Binance Adapter. Adapter xử lý rate limit, timeout và provider error, sau đó module validate, normalize, sắp xếp và deduplicate Candle. Closed Candle và Dataset reference được lưu idempotently; cache chỉ là lớp tăng tốc có TTL/version. Public response luôn dùng canonical contract.

5.2 Realtime Candle và phục hồi



Hình 5.1. Luồng realtime, reconnect và gap recovery

Một upstream stream có thể được chia sẻ cho nhiều subscriber cùng pair/timeframe. Khi mất kết nối, API không giả trạng thái connected mà phát RECONNECTING. Sau bounded exponential backoff, Market Module backfill từ closed Candle cuối đã xác nhận, hợp nhất vùng dữ liệu overlap và deduplicate theo provider–pair–timeframe–openTime. UI giữ snapshot hiện có, đánh dấu stale/recovering và reconcile sau khi kết nối lại.

5.3 Strategy, Backtest và Evaluation

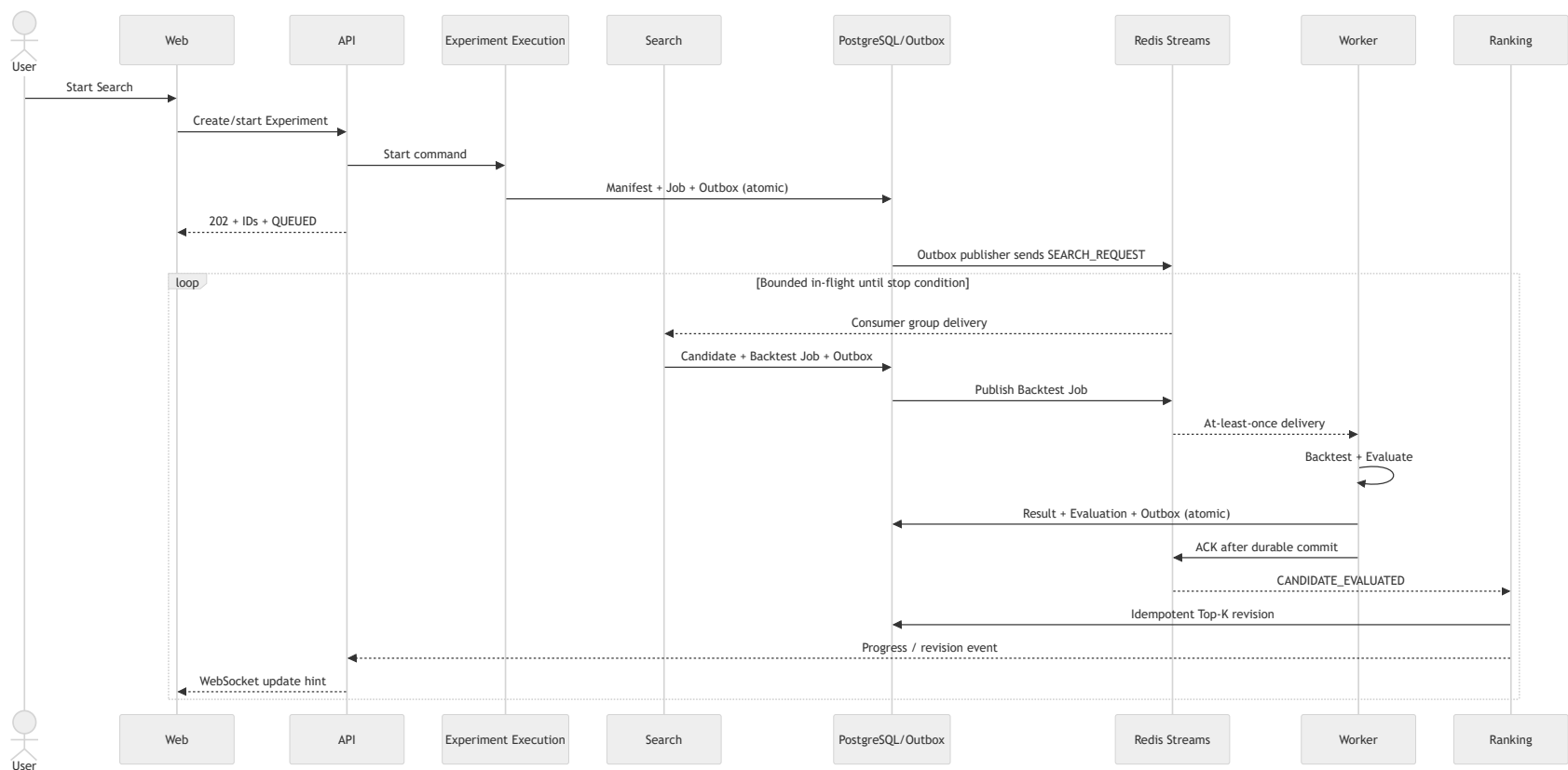


Hình 5.2. Luồng Strategy–Backtest–Evaluation

Worker chỉ bắt đầu khi resolve được Manifest, Dataset checksum và Strategy/Composite version. Registry validate parameters trước khi tạo instance. Backtester đưa ordered Candle

vào Strategy, diễn giải BUY/SELL/HOLD theo execution assumptions và tạo ordered Trade sequence. Evaluator tính metric độc lập; trường hợp không có Trade là business result hợp lệ. Nếu version hoặc checksum sai, Job thất bại có cấu trúc và không tạo partial Result.

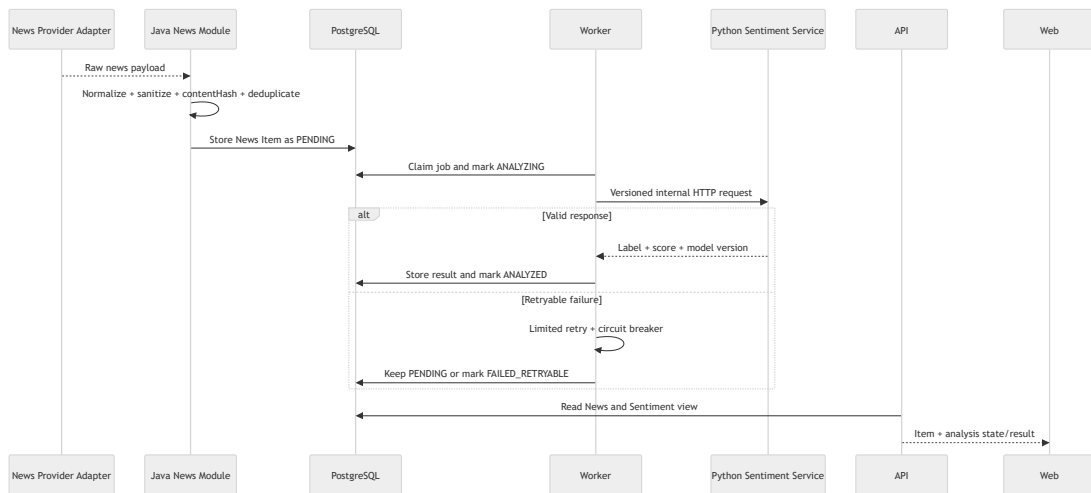
5.4 Search, Queue và Leaderboard



Hình 5.3. Luồng tạo Experiment, xử lý Candidate và cập nhật Top-K

Điểm quan trọng của luồng là ba transaction boundary: (1) tạo Manifest/Job/Outbox; (2) tạo Candidate/Backtest Job/Outbox cùng durable generator state; (3) commit Result/Evaluation/Outbox trước ACK. Coordinator duy trì bounded in-flight window và stop condition. Ranking xử lý CANDIDATE_EVALUATED idempotently, tạo Leaderboard revision và phát update hint; frontend luôn có thể tải authoritative snapshot từ REST.

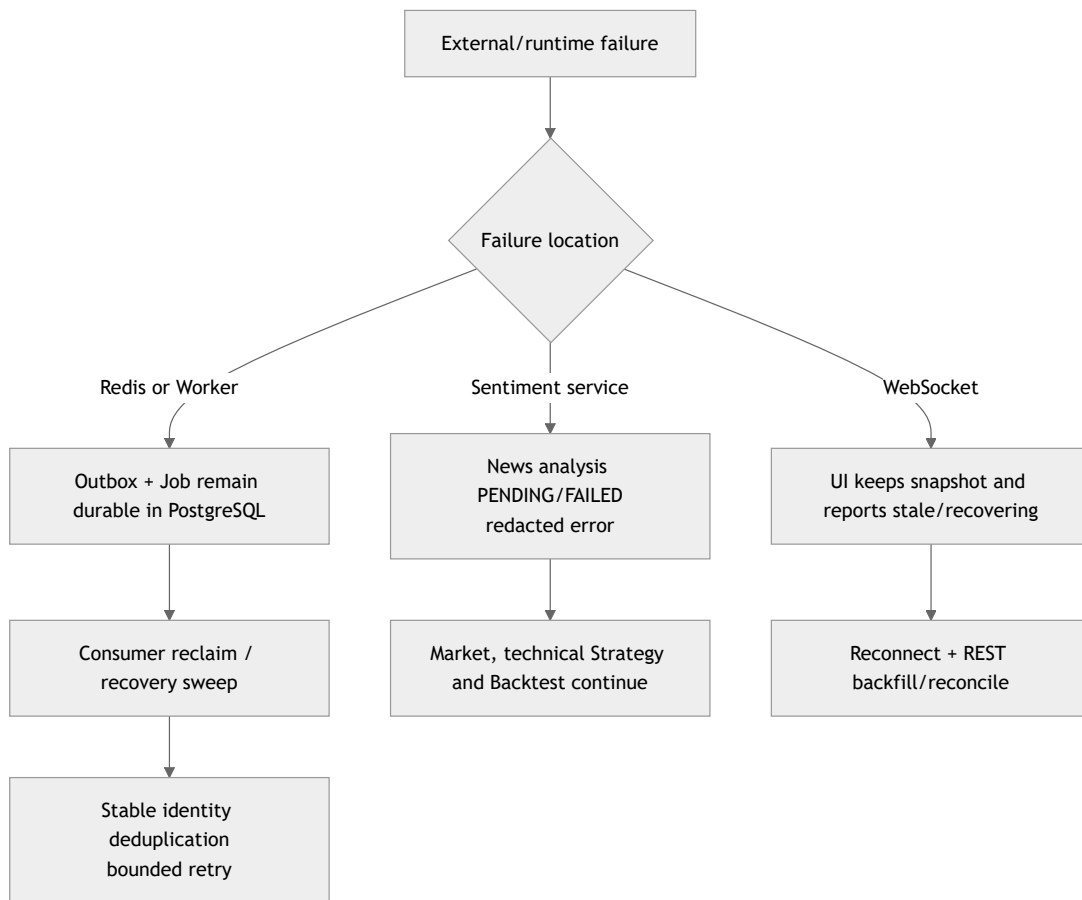
5.5 News và Sentiment



Hình 5.4. Luồng News collection và Sentiment Analysis

News Adapter chuyển raw payload thành News Item chuẩn, sanitize nội dung, tạo content hash và loại duplicate. Worker claim analysis lease và gọi versioned Sentiment contract. Response chỉ được persist khi đúng lease đang active và nằm trong range hợp lệ. Lỗi retryable giữ trạng thái minh bạch; UI vẫn đọc được News Item ngay cả khi sentiment tạm unavailable.

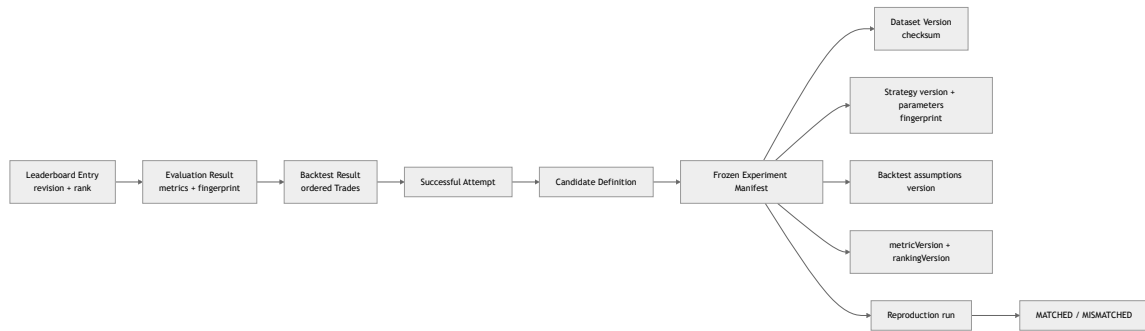
5.6 Failure isolation



Hình 5.5. Các failure boundary và cơ chế phục hồi

Ba nhánh failure được thiết kế độc lập. Redis/Worker failure dựa trên Outbox, pending reclaim và durable dedup. Sentiment failure chỉ làm degraded News Analysis. WebSocket failure giữ snapshot và dùng reconnect/reconciliation. PostgreSQL failure được xử lý fail-closed cho durable command; hệ thống không dùng Redis để giả một commit nghiệp vụ chưa xảy ra.

5.7 Leaderboard provenance



Hình 5.6. Đường truy vết từ Leaderboard đến input bất biến

Một entry được truy qua Evaluation Result, Backtest Result, successful Attempt, Candidate và frozen Manifest. Manifest dẫn đến Dataset checksum, Strategy version/parameters, assumptions, metric/ranking version và software identity. Khi reproduce, hệ thống tạo graph kết quả mới và trả verdict MATCHED/MISMATCHED; kết quả cũ vẫn bất biến.

Chương 6

Hiện thực hệ thống

6.1 Technology stack

Các phiên bản và capability trong mục này được đối chiếu trực tiếp với build scripts, package manifests, workflow và source code tại repository của nhóm [14]. Danh sách công nghệ, phiên bản và vai trò đã xác nhận được tổng hợp trong Bảng 6.1.

Bảng 6.1. Technology stack từ source/configuration

Lớp	Công nghệ	Vai trò
Backend	Java 21, Spring Boot 3.5.16, Gradle 8.14.5	API, Worker, capability module và build conventions.
Frontend	Next.js 16.3.4, React 19.1, TypeScript 5.9	Dashboard, authentication, realtime và feature routes.
Sentiment	Python 3.11–3.12, FastAPI 0.115.12, TensorFlow 2.19	Versioned inference runtime và health/readiness.
Persistence	PostgreSQL/Supabase, JDBC migrations	Durable business truth, Outbox và projection.
Async/cache	Redis 7, Redis Streams	Queue, Consumer Group, transient cache/progress.
Test	JUnit 5.12, ArchUnit 1.5, Pytest, Vitest, Playwright	Unit, contract, architecture, integration và browser test.
Delivery	Docker, Docker Compose, GitHub Actions	Sentiment image, local runtime và automated quality gates.

6.2 Strategy Engine và thư viện strategy

Strategy contract nhận context/candle batch và trả decision chuẩn. Descriptor mô tả plugin ID, semantic version, display name, parameter rules, supported signal và search range hint. Registry từ chối identity trùng. Trusted catalog hiện đăng ký bốn plugin:

- **MA Crossover**: so sánh fast/slow moving average và yêu cầu fast period nhỏ hơn slow period;
- **RSI**: sử dụng period cùng ngưỡng oversold/overbought;
- **Bollinger Bands**: dùng moving window và độ lệch chuẩn để xác định band;
- **Support/Resistance**: nhận diện vùng giá trong lookback window và tolerance.

Composite Strategy chứa ordered component snapshots và policy version. Majority Vote chọn tín hiệu chiếm đa số; Weighted Vote nhân signal encoding với trọng số và threshold. Tie hoặc score nằm trong neutral band tạo HOLD. Policy nằm ngoài Strategy implementation để có thể thay quy tắc combine mà không sửa từng plugin.

6.3 Search Space và điều phối Experiment

Random Search dùng seed/state bất biến để có thể tiếp tục và tái tạo candidate sequence. Search Space hỗ trợ strategy pool, parameter range và constraint; composite refill giữ bounded in-flight thay vì đổ toàn bộ candidate vào queue. Stop condition hữu hạn theo maximum candidates và trạng thái terminal. Durable decision/fence ngăn hai Coordinator đồng thời cấp phát cùng một slot sau retry/recovery.

6.4 Backtesting assumptions và metrics

Backtest không giả định một signal chính là giao dịch tức thì. Execution assumptions quy định thời điểm vào/thoát, capital, fee và dữ liệu được phép nhìn thấy nhằm tránh look-ahead bias. Strategy chỉ được đánh giá trên ordered Candle và evaluation time. Result lưu initial/final capital cùng ordered Trade; Evaluation tách khỏi Strategy implementation. Ý nghĩa và giới hạn diễn giải của từng metric được nêu tại Bảng 6.2.

Bảng 6.2. Các metric đánh giá bắt buộc

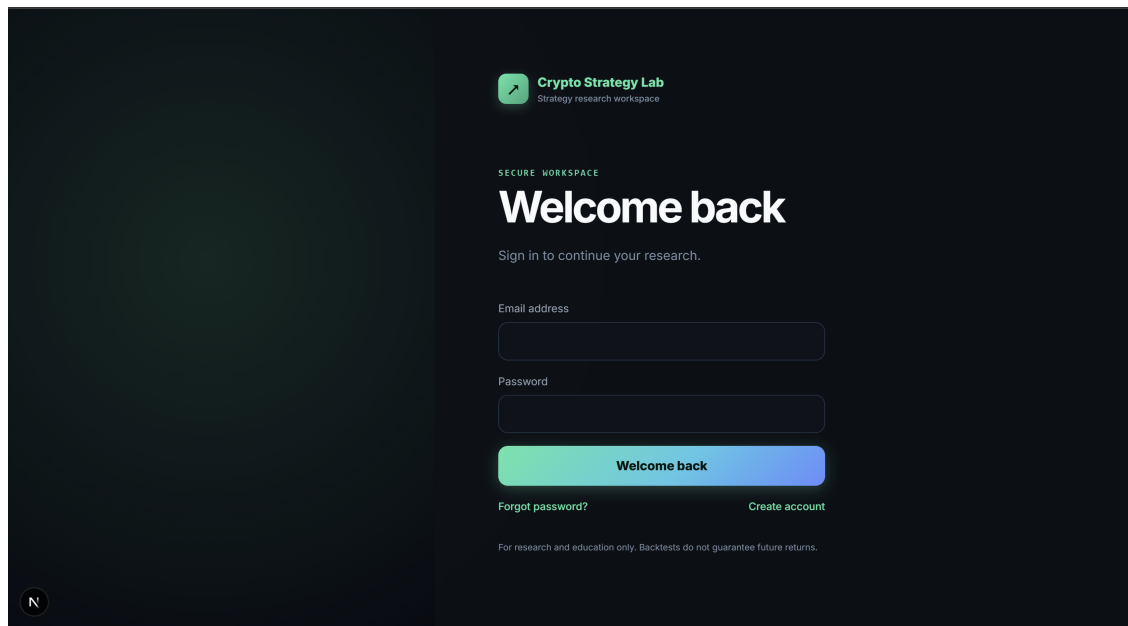
Metric	Ý nghĩa	Lưu ý diễn giải
Total Return	Tỷ lệ thay đổi capital sau backtest.	Không đủ để kết luận strategy ổn định.
Win Rate	Tỷ lệ trade có profit dương.	Cần đọc cùng payoff và số lượng trade.
Maximum Drawdown	Sụt giảm lớn nhất từ đỉnh equity đến đáy tiếp theo.	Đại diện rủi ro đường đi, không chỉ kết quả cuối.
Number of Trades	Số giao dịch hoàn tất.	Mẫu quá nhỏ làm metric khó tổng quát.
Overall Score	Hàm xếp hạng có version từ các input metric.	Phải lưu ranking/metric version và deterministic tie-break.

6.5 Leaderboard và dữ liệu bất biến

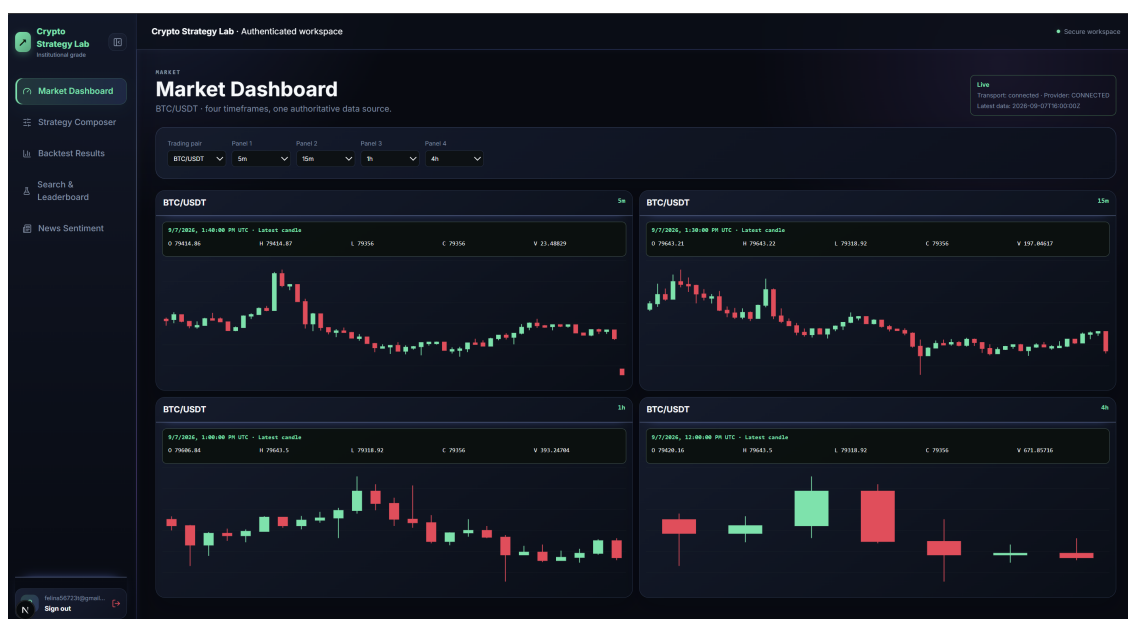
Leaderboard là read projection theo Experiment. Mỗi revision ghi Top-K và ordered Entry; tie-break ổn định để cùng input tạo cùng order. Entry liên kết Evaluation Result thay vì copy một score không nguồn gốc. Khi retry Candidate, unique identity và idempotency ngăn tạo nhiều Result/Entry cho cùng business outcome.

6.6 Giao diện người dùng thực tế

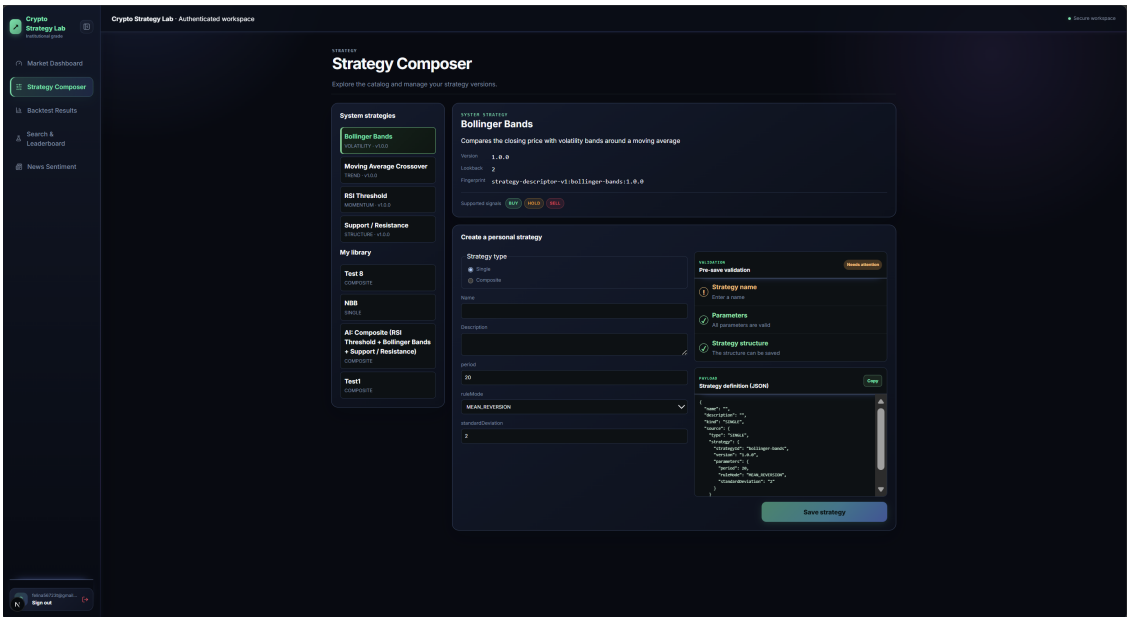
Giao diện web đã được triển khai theo các nhóm chức năng chính gồm xác thực người dùng, theo dõi thị trường, xây dựng chiến lược, thực hiện backtest, tìm kiếm chiến lược, quản lý bảng xếp hạng và phân tích cảm xúc tin tức. Chức năng xác thực và quản lý phiên thuộc lớp nền tảng; các mô-đun nghiệp vụ giao tiếp với backend thông qua API có kiểu dữ liệu rõ ràng và client thời gian thực. Các hình bên dưới là ảnh chụp giao diện thực tế của ứng dụng web, thể hiện luồng sử dụng từ đăng nhập, cấu hình chiến lược và backtest đến theo dõi kết quả tìm kiếm, bảng xếp hạng và sentiment.

**Hình 6.1.** UI — Log In & Register

Nguồn: Ảnh chụp giao diện thực tế của ứng dụng web.

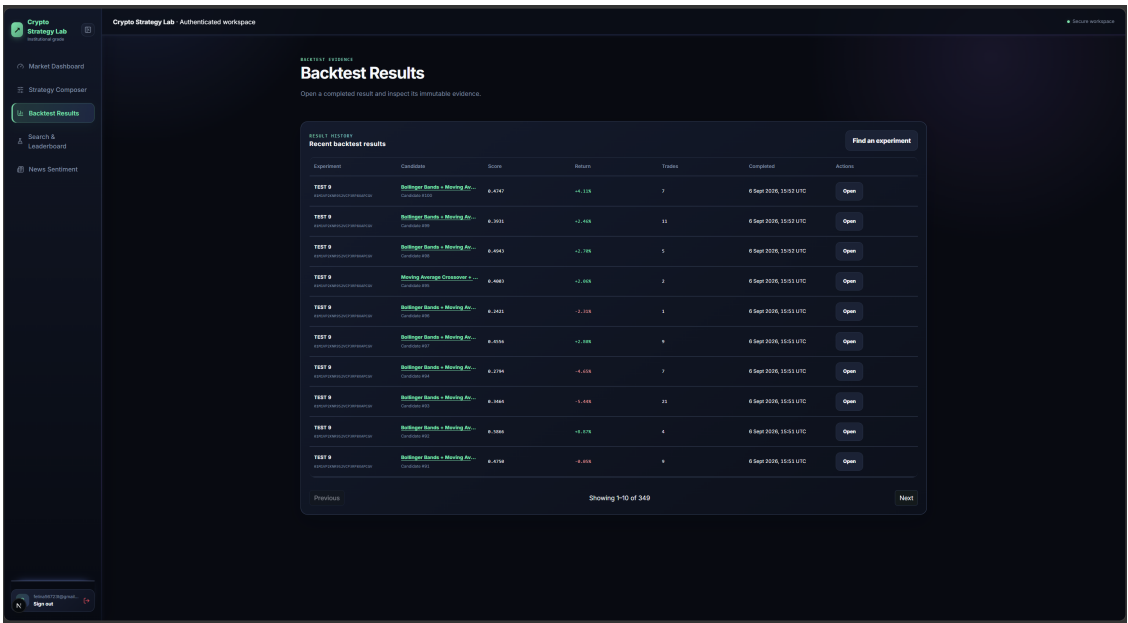
**Hình 6.2.** UI — Market Dashboard nhiều timeframe

Nguồn: Ảnh chụp giao diện thực tế của ứng dụng web.



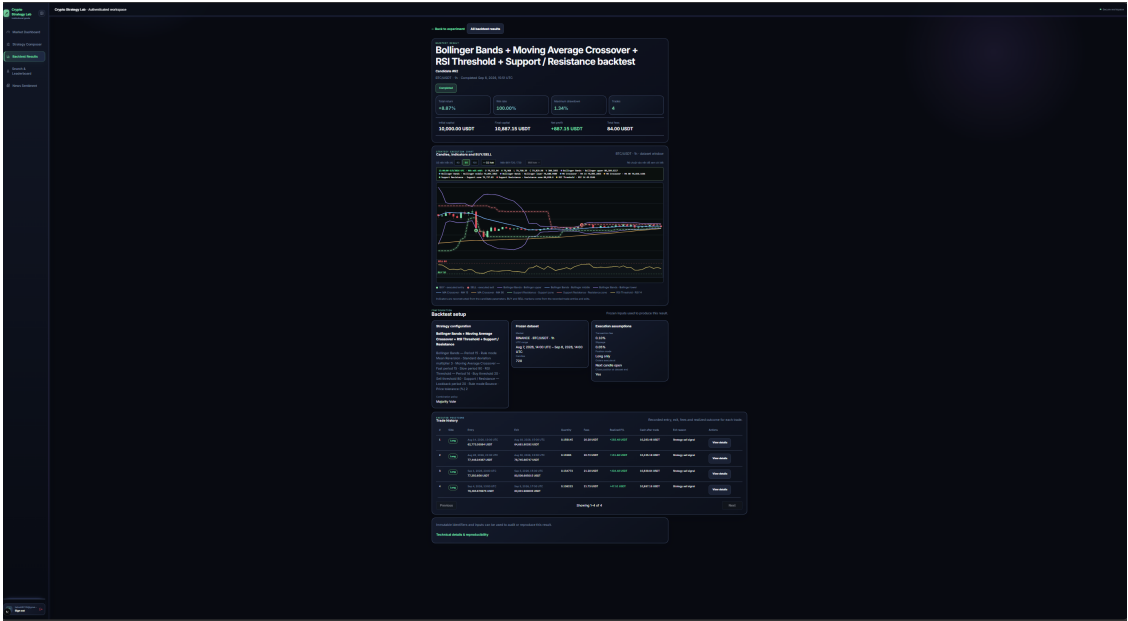
Hình 6.3. UI — Strategy Composer

Nguồn: Ảnh chụp giao diện thực tế của ứng dụng web.



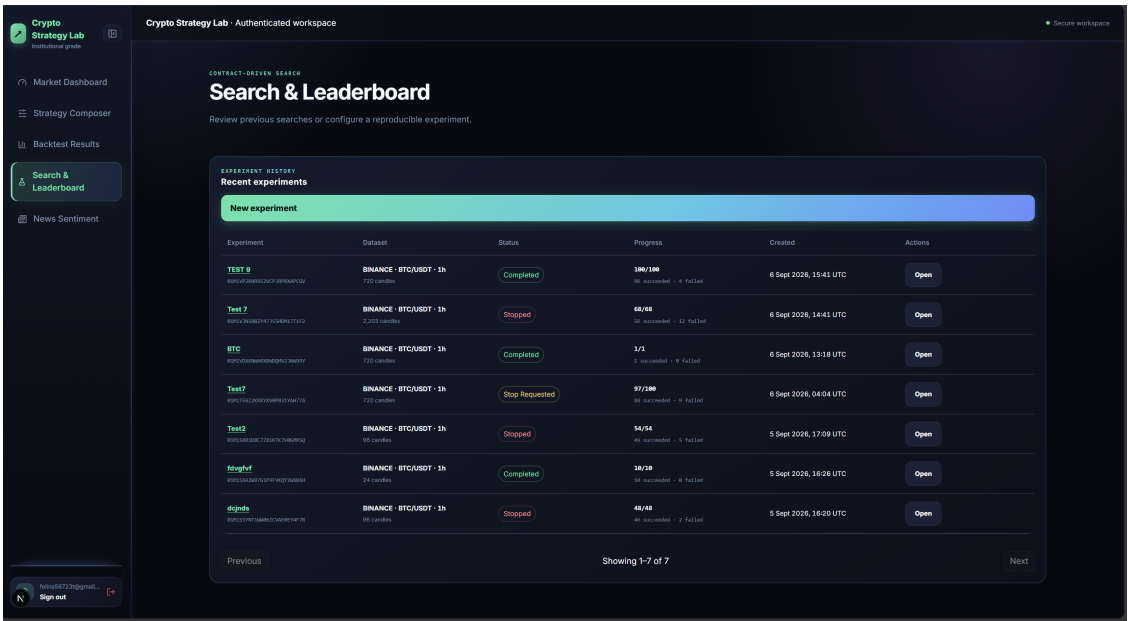
Hình 6.4. UI — Backtest Result và Trade detail

Nguồn: Ảnh chụp giao diện thực tế của ứng dụng web.



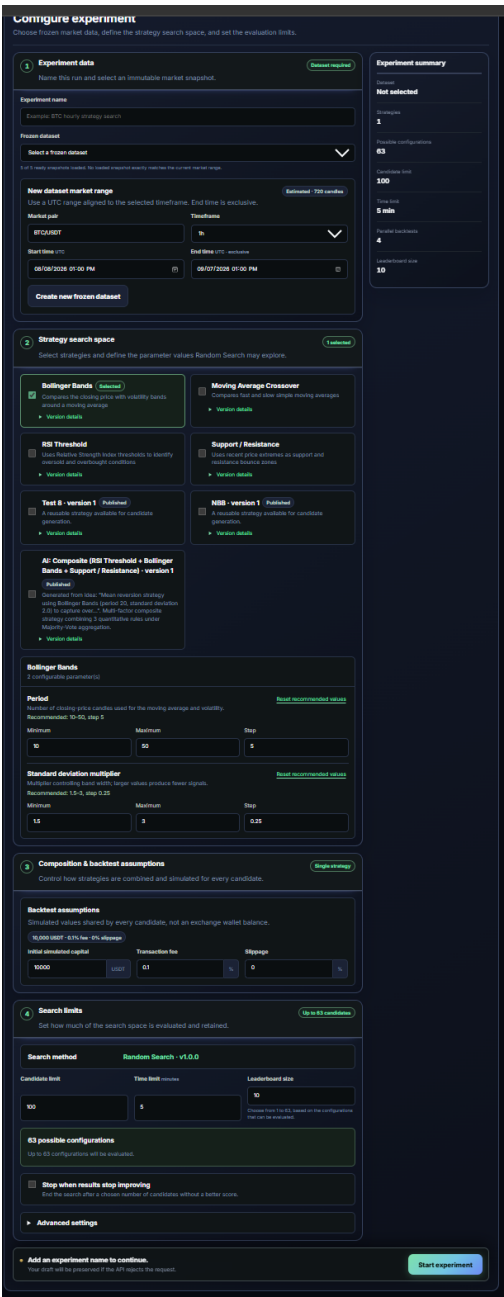
Hình 6.5. UI — Backtest detail

Nguồn: Ảnh chụp giao diện thực tế của ứng dụng web.



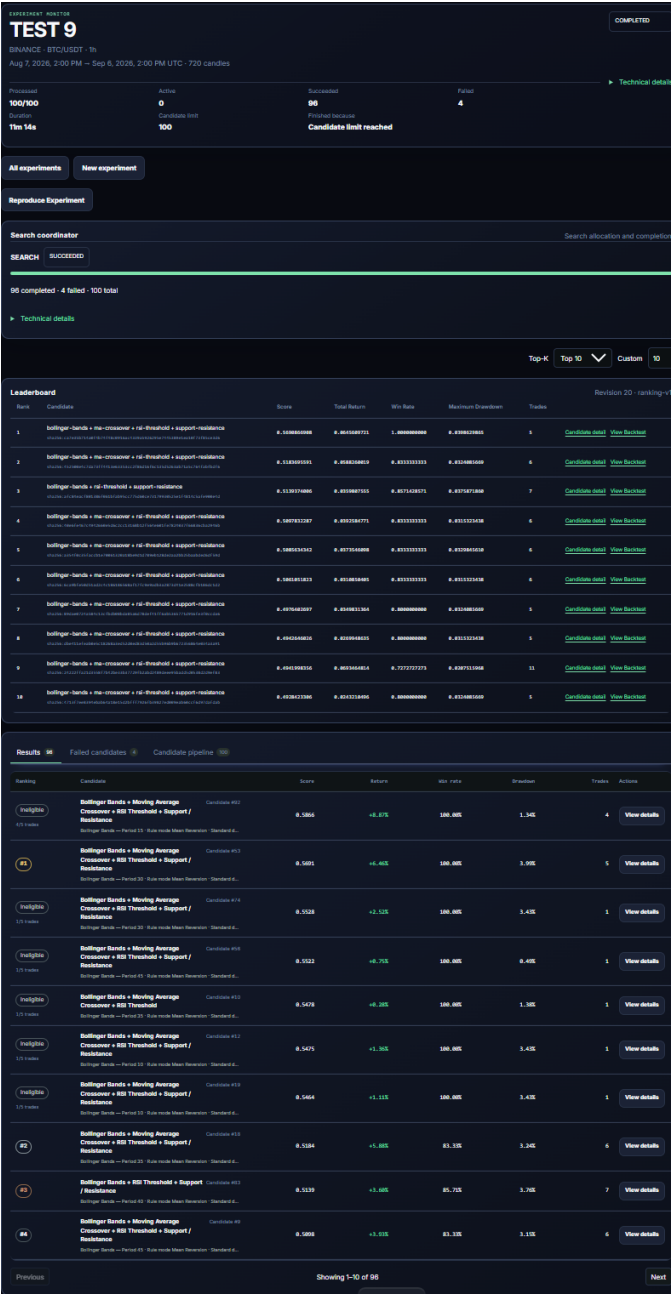
Hình 6.6. UI — Search results

Nguồn: Ảnh chụp giao diện thực tế của ứng dụng web.



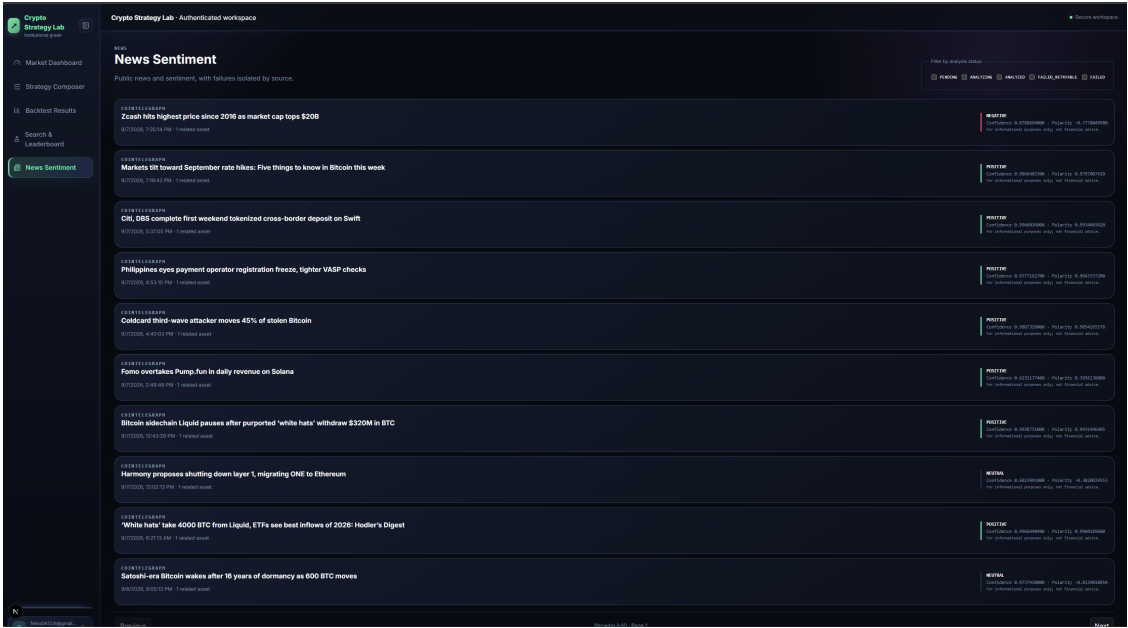
Hình 6.7. UI — Search configuration

Nguồn: Ảnh chụp giao diện thực tế của ứng dụng web.



Hình 6.8. UI — Search process and leaderboard

Nguồn: Ảnh chụp giao diện thực tế của ứng dụng web.



Hình 6.9. UI — New Sentiment

Nguồn: Ảnh chụp giao diện thực tế của ứng dụng web.

Chương 7

Reliability, Scalability, Security và Observability

7.1 Backpressure và bounded work

API không đợi Search hoàn tất mà trả resource ID sau durable acceptance. Coordinator cấp phát candidate theo bounded in-flight window; Worker dùng fixed thread pool và semaphore/concurrency configuration. Batching giảm overhead nhưng kích thước batch phải được giữ trong giới hạn memory, DB connection và job timeout. Đây là cách làm backlog hữu hạn và quan sát được thay vì sinh vô hạn task vào Redis.

7.2 Scale ngang Worker

Các Worker dùng cùng stream/group nhưng consumer name riêng. Redis Consumer Group phân phối message; PostgreSQL giữ Job/Attempt/Result. Scale từ một lên nhiều Worker không yêu cầu sửa Strategy, Backtester hay Evaluation contract. Bottleneck có thể chuyển từ CPU sang Redis, database pool, dataset loading hoặc write contention; vì vậy scale-out phải đi cùng measurement queue lag, throughput, duplicate count và DB utilization.

Kịch bản 100.000 backtest không đồng nghĩa đẩy 100.000 message cùng lúc. Kiến trúc dự kiến phân trang/generate theo window, ưu tiên bounded queue, giới hạn concurrency toàn Experiment và có stop/cancel semantics. Nếu workload vượt một host, Worker có thể tăng instance; nếu database trở thành bottleneck, cần tối ưu batch persistence, index, connection budget hoặc partitioning trước khi thêm công nghệ mới.

7.3 Retry, recovery và deduplication

Các failure mode, tactic, invariant và tín hiệu quan sát tương ứng được hệ thống hóa trong Bảng 7.1.

Bảng 7.1. Reliability tactics theo failure mode

Failure	Tactic	Invariant	Quan sát
Publish Redis lỗi	Transactional Outbox và republish.	Intent đã commit không bị mất.	publish attempt, last error.
Worker crash	Pending reclaim/recovery sweep.	Cùng Job có stable identity.	pending age, consumer/attempt.
Duplicate delivery	Durable processed record/idempotent write.	Tối đa một business Result hợp lệ.	duplicate count, message ID.
Lỗi transient	Bounded retry và exponential backoff.	Không retry vô hạn.	attempt no., next retry, error code.
Lỗi permanent	FAILED/DLQ và redacted reason.	Không ACK như thành công.	DLQ depth, failure status.
Redis mất tạm	Rebuild transport/progress từ PostgreSQL/Outbox.	Redis không là business truth.	queue health, recovery lag.
Sentiment down	Timeout, circuit breaker, degraded lifecycle.	Market/technical flow không bị chặn.	circuit state, analysis status.
WebSocket mất	Reconnect, resubscribe, REST reconcile.	Client không dùng stale revision.	connection state, revision.

7.4 Observability

Một lỗi background không thể chỉ xuất hiện trong console. Request, Experiment, Candidate, Job, Attempt và message cần correlation identity để truy một flow xuyên API, Outbox, Redis và Worker. Progress snapshot tối thiểu gồm queued/running/completed/failed count, best score, revision và terminal reason. Liveness chỉ cho biết process sống; readiness cho biết runtime có đủ dependency bắt buộc để nhận work.

Các target observability trong tài liệu là khả năng nhìn thấy progress trong vài giây, trace theo identity và phân biệt retryable/permanent failure. Repository có Spring Boot Actuator, structured domain state và WebSocket progress; chưa tuyên bố có production metrics backend hoặc distributed tracing platform hoàn chỉnh.

7.5 Security boundary

Web đăng nhập qua Supabase Auth và gửi JWT đến API. Backend xác thực issuer, JWKS và audience, sau đó kiểm tra ownership theo user cho Strategy, Experiment và Result. WebSocket

dùng ticket ngắn hạn thay vì đặt access token dài hạn trong URL. Browser chỉ chứa biến NEXT_PUBLIC; database password, service-role key và Sentiment token không được phát xuống client.

Sentiment endpoint là internal boundary có service token; Python không tự fetch URL do người dùng cung cấp. External provider payload được validate/sanitize; public error không lộ stack trace hoặc credential. Migrations là forward-only source of truth và secret không được commit vào repository/evidence.

Chương 8

Đánh giá kiến trúc và Architecture Proof

8.1 Phương pháp ATAM-lite

ATAM-lite được dùng như một “crash test” cho kiến trúc: đưa scenario thay đổi hoặc failure vào, xác định sensitivity point, quan sát response và ghi trade-off [3, 5]. Mục tiêu không phải chấm sơ đồ đẹp mà là chứng minh boundary tồn tại trong code và hành vi. Kết quả đối chiếu giữa scenario, tactic, measure và evidence được trình bày tại Bảng 8.1.

Bảng 8.1. Architecture proof theo scenario

Scenario	Decision/tactic	Response và measure	Evidence	Status
Thêm MACDStrategy	Plugin/Registry và public contract.	Fixture plugin được thêm mà downstream không phụ thuộc implementation.	72 foundation tests; extension test.	VERIFIED
Đổi Search Generator	Strategy Generator Registry.	Generator thay thế vẫn trả Candidate Definition; Backtester/Evaluator không đổi.	Contract/ architecture tests; chưa có production Domain- guided hoặc Genetic Generator.	PARTIAL
Thay Market Provider	MarketData Provider Adapter.	Binance và Fixture dùng canonical Candle; frontend contract không đổi.	Adapter tests; Fixture không phải exchange thực tế thứ hai.	PARTIAL

Scenario	Decision/tactic	Response và measure	Evidence	Status
Tăng Worker, duplicate delivery	Streams Consumer Group, reclaim và durable dedup.	Reclaim đúng message; accepted outcome giữ bằng 1.	Redis thật và recovery test.	VERIFIED
Sentiment Service failure	Timeout/retry/circuit breaker và isolation.	Backend/UI controlled pass; external model process restart pass; chưa có authenticated LIVE outage journey xuyên API-Worker-Web.	Failure-recovery evidence.	PARTIAL
Reproduce Top-K	Frozen Manifest/checksum/fingerprint.	Source/target graph và verdict MATCHED, không sửa Result gốc.	PostgreSQL reproduction evidence.	VERIFIED

8.2 Benchmark concurrency 1 và 3

Benchmark F014 chạy standalone trên commit sạch, Apple M2, Java 21, 12 candidate mỗi run và 2.000 candle một phút cho mỗi candidate. Mỗi profile chạy ba lần, đổi thứ tự để giảm lợi thế JIT/cache. Kết quả raw được giữ trong Bảng 8.2.

Bảng 8.2. Kết quả benchmark Backtest concurrency 1 so với 3

Run	Concurrency	Compute (ms)	Total (ms)	p95 (ms)	Candidate/s	Speedup	Duplicate
1	1	129.350	129.443	15.205	92.771	2.527×	0
1	3	51.181	51.239	13.343	234.462		0
2	1	71.633	71.697	7.343	167.521	1.697×	0
2	3	42.201	42.260	11.416	284.353		0
3	1	66.290	66.360	6.473	181.023	2.534×	0
3	3	26.164	26.261	7.257	458.638		0

Nguồn: *docs/evidence/f014/performance.md, profile IN_PROCESS_BACKTEST.*

Median standalone là 2.527×, 36/36 candidate hoàn tất cho mỗi profile, không timeout và không duplicate identity. Tuy nhiên, đây là compute benchmark trong một JVM; không gồm HTTP, Redis, PostgreSQL, network, startup hoặc nhiều Worker process. Khi chạy cùng workload npm ci, median chỉ đạt 1.836×, cho thấy kết quả nhạy với host contention. Báo cáo không dùng số liệu này để tuyên bố horizontal scaling hoặc production SLA.

8.2.1 Evidence scale F015: 100, 1.000 và 10.000 candidate

Sau benchmark compute F014, nhóm tiếp tục kiểm tra khả năng cấp phát và xử lý candidate ở ba mức tải khác nhau. Các phép đo này có phạm vi khác nhau và không được cộng gộp thành một production benchmark duy nhất.

Bảng 8.3. Tổng hợp evidence scale F015

Quy mô	Phạm vi kiểm tra	Kết quả chính	Giới hạn diễn giải
100 candidate	Durable integration trên PostgreSQL 17.	Persist đúng 100 Candidate, 100 fingerprint và 100 Backtest Job; peak active bằng 4, không phát hiện duplicate.	Dependency-backed integration; không phải production traffic.
1.000 candidate	Controlled comparative Worker profile.	Cấu hình rộng nhanh hơn khoảng 5.39×; hai profile tạo cùng ordered candidate/outcome hash.	Controlled in-process profile; không đại diện end-to-end latency ngoài production.
10.000 candidate	Bounded allocation và backpressure profile.	Hoàn tất 10.000/10.000 work item; peak active 6/8, pending tối đa 32, heap tăng khoảng 70 MB và dưới giới hạn 256 MiB.	Không phải 10.000 financial Backtest đầy đủ qua toàn bộ external infrastructure.

Nguồn: `docs/evidence/f015/100-candidate-integration.md`, `1000-candidate-performance.md` và `10000-candidate-backpressure.md`.

Evidence hiện tại chứng minh bounded generation, durable allocation, backpressure và deterministic outcome đến mức 10.000 candidate trong các profile đã mô tả. Nhóm chưa tuyên bố đã chạy 100.000 full production Backtest, horizontal scaling nhiều host hoặc production SLA.

8.3 Tổng hợp evidence hiện tại

Theo ma trận evidence F014, trong 23 tiêu chí cốt lõi có 5 **VERIFIED**, 16 **PARTIAL** và 2 **BLOCKED**. Các evidence F015 bổ sung kết quả scale nhưng không tự động chuyển những tiêu chí cần authenticated LIVE evidence sang **VERIFIED**. Bảng tự đánh giá trong Excel là mức nhóm tự khai và không thay thế evidence status. Phân bố được tổng hợp tại Bảng 8.4.

Bảng 8.4. Tổng hợp trạng thái kiểm chứng theo nhóm tiêu chí

Nhóm	Verified	Partial	Blocked	Nhận xét
Kiến trúc	3	4	0	Plugin, Worker recovery và reproduction có evidence mạnh; replaceability, realtime và isolation vẫn chủ yếu được kiểm tra trong controlled profile.
Chức năng	1	9	0	Bốn strategy được kiểm chứng; các flow còn lại có code/test nhưng cần bổ sung LIVE IDs, timestamp hoặc screenshot từ cùng một phiên chạy.
Hồ sơ	1	2	1	README và ADR đầy đủ; link video/report đã khai nhưng quyền truy cập và nội dung video cần được đối chiếu trước khi nâng trạng thái.
Demo	0	1	1	Controlled journey đã pass; full authenticated LIVE journey chỉ được xác nhận sau khi video thể hiện đủ flow và các resource ID liên quan.
Mở rộng	0	0	0	Giữ NO CLAIM trong evidence matrix; các mục điểm cộng được khai riêng và do giảng viên đánh giá.

Chương 9

Cài đặt, build, kiểm thử và CI/CD

9.1 Môi trường local

Máy phát triển cần Git, JDK 21, Node.js 22, Docker và PostgreSQL/Supabase. Python 3.11–3.12 chỉ bắt buộc khi chạy Sentiment ngoài Docker. Các biến môi trường được tạo từ `.env.example`; secret và credential thật không được đưa vào báo cáo hoặc source control.

Thứ tự khởi động khuyến nghị là: áp dụng migration PostgreSQL, chạy Redis, chạy Sentiment, API, Worker và cuối cùng Web. API mặc định có thể dùng fixture provider; cấu hình Binance chỉ bật khi cần dữ liệu live. Mỗi Worker phải có consumer name riêng. Hướng dẫn đầy đủ và lệnh PowerShell/macOS/Linux được duy trì tại:

[docs/SETUP.md](#) — Hướng dẫn Setup, Cài đặt và Build

9.2 Build và test strategy

Root check chạy verification của toàn bộ Java subproject, architecture tests và build logic. Integration test có external database là task riêng để tránh biến unit gate thành phụ thuộc môi trường. Web dùng một quality gate gồm format check, ESLint, TypeScript, Vitest và Next.js production build. Sentiment có Pytest contract/unit suite và Docker image chứa extra ML. Các lệnh tiêu biểu và mục đích của từng quality gate được tổng hợp tại [Bảng 9.1](#).

Bảng 9.1. Các quality gate chính

Phạm vi	Lệnh tiêu biểu	Mục đích
Java toàn bộ	<code>./gradlew clean check</code>	Unit, contract, module và build verification.
Architecture	<code>./gradlew :architecture-tests:test</code>	Dependency, purity, cycle và extension boundary.
PostgreSQL	Các task <code>*IntegrationTest</code>	Migration, persistence contract và reproduction graph.
Web	<code>npm run check</code>	Format, lint, typecheck, Vitest và build.
Browser	<code>npm run test:e2e</code>	Journey/realtime/recovery ở profile phù hợp.
Sentiment	<code>python -m pytest</code>	Contract, model runtime và API behavior.
Performance	Worker/API controlled task	Throughput, timeout, duplicate và target cụ thể.

9.3 CI/CD hiện có

GitHub Actions workflow CI chạy Java tests và Sentiment contract/Python tests khi push main hoặc mở pull request. Workflow Web Foundation chạy khi thay đổi frontend, bao gồm `npm audit --audit-level=high` và `npm run check`. Repository chưa cung cấp production deployment URL hoặc pipeline release/deploy hoàn chỉnh, vì vậy báo cáo chỉ gọi đây là Continuous Integration, không tuyên bố Continuous Deployment.

Chương 10

Kết luận và hướng phát triển

10.1 Kết luận

Đồ án đã chuyển bài toán từ một tập indicator rời rạc thành một experiment platform có boundary. Strategy mới đi qua Plugin/Registry; provider đi qua Adapter; generator đi qua replaceable contract; tác vụ dài đi qua durable Queue–Worker; Result đi kèm frozen provenance. Kiến trúc không loại bỏ complexity mà đặt complexity đúng nơi: duplicate được xử lý ở message boundary, ML failure ở Sentiment boundary, provider drift ở adapter và version drift ở Manifest.

Điểm quan trọng nhất là sự liên kết giữa quyết định và bằng chứng. Modular Monolith được bảo vệ bởi module/architecture tests; Worker recovery có Redis reclaim/dedup test; parallel backtest có benchmark công bố điều kiện; reproducibility có graph và verdict. Đồng thời, những phần chưa có authenticated LIVE evidence được giữ trạng thái Partial/Blocked thay vì được che bằng prototype hoặc fixture.

10.2 Bài học kiến trúc

Một kiến trúc tốt không được xác định bởi số lượng công nghệ. Nó được xác định bởi khả năng trả lời: điều gì sẽ thay đổi, điều gì có thể hỏng, boundary nào hấp thụ thay đổi và bằng chứng nào cho thấy hệ thống vẫn đúng. Với Crypto Strategy Lab, các driver này dẫn đến một hệ thống có thể tiếp tục phát triển mà không biến mọi yêu cầu mới thành thay đổi xuyên toàn bộ codebase.

Tài liệu tham khảo

- [1] Nhóm giảng viên môn Kiến trúc phần mềm, *Crypto Strategy Lab – Nền tảng phân tích, kết hợp và đánh giá chiến lược giao dịch Crypto*, Đề bài đồ án cuối kỳ, 2026.
- [2] Trần Văn Quý, *Kiến trúc cho Crypto Strategy Lab: Từ “chạy được” đến một nền tảng có thể thay đổi, mở rộng và kiểm chứng*, tài liệu seminar môn Kiến trúc phần mềm, 2026.
- [3] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed., Addison-Wesley, 2021.
- [4] P. Clements et al., *Documenting Software Architectures: Views and Beyond*, 2nd ed., Addison-Wesley, 2010.
- [5] H. Cervantes and R. Kazman, *Designing Software Architectures: A Practical Approach*, Addison-Wesley, 2016.
- [6] S. Brown, *The C4 Model for Visualising Software Architecture*. <https://c4model.com/>. Truy cập tháng 9 năm 2026.
- [7] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- [8] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.
- [9] M. Fowler et al., *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.
- [10] C. Richardson, *Microservices Patterns*, Manning, 2019.
- [11] M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
- [12] Redis Ltd., *Redis Streams documentation*. <https://redis.io/docs/latest/development/data-types/streams/>. Truy cập tháng 9 năm 2026.
- [13] PostgreSQL Global Development Group, *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>. Truy cập tháng 9 năm 2026.

- [14] Nhóm 09, *Crypto Strategy Platform source code and architecture documentation*. <https://github.com/PhamVanMinhBinhThuan/crypto-strategy-platform>. Truy cập tháng 9 năm 2026.

Chương A

Danh mục Architecture Decision Record

Bảng A.1 liệt kê 17 ADR đã được đối chiếu trong repository cùng trạng thái và driver chính tại thời điểm lập báo cáo. Mỗi ADR ghi Context, Decision, Alternatives, Consequences và evidence/verification. Quyết định mới không sửa lịch sử ADR cũ; khi một quyết định bị thay thế, nhóm tạo ADR mới và đánh dấu ADR trước là *Superseded*.

Bảng A.1. Danh mục ADR tại thời điểm lập báo cáo

ADR	Quyết định	Driver chính	Status
0001	Modular Monolith	Modifiability	Accepted
0002	Module boundary và dependency direction	Maintainability	Accepted
0003	Market Data Provider Adapter	Replaceability	Accepted
0004	WebSocket realtime và recovery	Realtime	Accepted
0005	Strategy Plugin/Registry	Extensibility	Accepted
0006	Queue–Worker cho Backtest/Search	Scalability	Accepted
0007	PostgreSQL/Redis ownership	Reliability	Accepted
0008	Sentiment Service boundary	Isolation	Accepted
0009	Reproducible Experiment	Reproducibility	Accepted
0010	Strategy Generator contract	Replaceability	Accepted
0011	Supabase Auth/user ownership	Security	Accepted
0012	User Strategy và durable Job ownership	Correctness	Accepted
0013	Backtest execution integration	Correctness	Accepted
0014	Experiment Execution Orchestrator	Maintainability	Accepted
0015	Standalone Backtest aggregate	Correctness	Proposed
0016	Durable Search Coordinator	Reliability	Accepted
0017	Composite Search Space/refill	Scalability	Accepted

Chương B

Requirement Traceability Matrix

Bảng B.1 nối yêu cầu/câu hỏi kiến trúc với decision, module, evidence và vị trí giải thích trong báo cáo.

Bảng B.1. Ánh xạ yêu cầu–kiến trúc–evidence

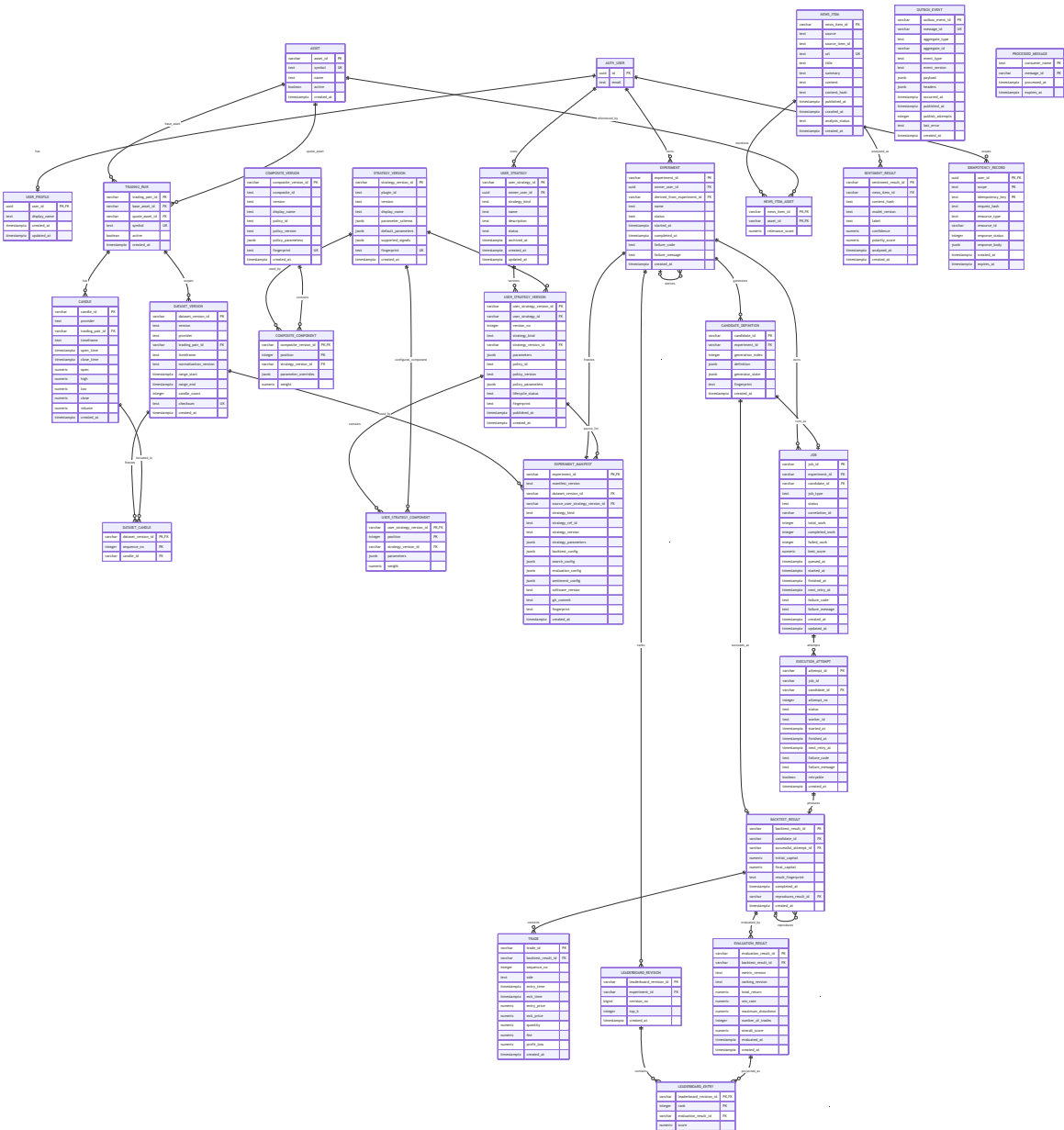
Yêu cầu/câu hỏi	Decision/module	Evidence chính	Mục báo cáo
Thêm Strategy mới	ADR-0005; strategy-core/strategies	Strategy extension/foundation tests	4, 8
Đổi Search algorithm	ADR-0010; search Registry	Generator replaceability tests	4, 8
Thêm Market Provider	ADR-0003; market-data port	Binance/Fixture contract tests	5, 8
Scale từ 100 đến 10.000 candidate; 100.000 là hướng mở rộng	ADR-0006/0016/0017	F014 compute benchmark và F015 durable/bounded scale evidence	7, 8
News/Sentiment lỗi không làm Chart dừng	ADR-0008; failure boundary	Controlled isolation và external restart evidence	5, 8
Binance disconnect	ADR-0004; realtime recovery	Reconnect/reconciliation tests	5
Duplicate/retry/order	ADR-0006/0007	Redis reclaim/dedup evidence	7
Leaderboard provenance	ADR-0009; frozen Manifest	Reproduction MATCHED graph	5, 8

Yêu cầu/câu hỏi	Decision/module	Evidence chính	Mục báo cáo
Architecture document	C4 + module + dynamic/data views	docs/architecture và ArchUnit	3
Install/Run	README + SETUP	Repository commands/config	9

Chương C

ERD và Event Catalog

Hình C.1 mở rộng mô hình khái niệm thành schema nghiệp vụ, còn Bảng C.1 mô tả các message quan trọng tại asynchronous boundary.



Hình C.1. ERD đầy đủ của PostgreSQL business schema

Nguồn: docs/database/drawio-erd.mmd; migration trong supabase/migrations.

Bảng C.1. Event/message catalog rút gọn

Message/event	Producer	Consumer/ý nghĩa	Correctness rule
SEARCH_REQUEST v1	API/Execution Outbox	Search Coordinator bắt đầu/resume run.	Stable run/job identity, version fence.
BACKTEST_JOB	Search/Execution Outbox	Backtest Worker xử lý Candidate.	At-least-once; ACK sau commit; dedup Result.

Message/event	Producer	Consumer/ý nghĩa	Correctness rule
CANDIDATE_ EVALUATED	Worker Outbox	Ranking và Coordinator.	Hai Consumer Group độc lập; idempotent score/ reconcile.
PROGRESS/ STATUS	Worker/ Execution	API WebSocket và UI.	Chỉ là update hint; REST snapshot authoritative.
LEADERBOARD_ REVISION	Ranking	API/Web cập nhật Top-K.	Revision tăng đơn điều; client bỏ stale.
NEWS_ANALYSIS	News/Worker	Sentiment processing lifecycle.	Lease/attempt active mới được complete.
DEAD_LETTER	Worker	Operator/recovery workflow.	Redact secret; lưu reason và original identity.

Chương D

Bảng tự đánh giá và đóng góp thành viên

D.1 Nguyên tắc đọc bảng

Bảng D.1. Mức tự đánh giá của nhóm theo rubric

STT	Tiêu chí	Tự đánh giá	Ghi chú evidence
1	Khả năng mở rộng Strategy/Plugin	4/4	Có contract, registry, architecture test và extension proof.
2	Tách trách nhiệm và giảm coupling	4/4	Có module boundary và architecture tests; evidence tổng vẫn còn Partial.
3	Khả năng thay thế thành phần	4/4	Đã chứng minh boundary bằng Fixture; chưa có production Generator/ exchange thứ hai.
4	Scalability và Performance	4/4	Có F014 compute benchmark và F015 controlled scale evidence.

STT	Tiêu chí	Tự đánh giá	Ghi chú evidence
5	Realtime và Multi-timeframe	4/4	Có code và controlled test; Binance LIVE evidence cần đối chiếu trong video.
6	Reliability và Observability	4/4	Redis recovery đã verified; full LIVE failure-isolation journey cần đối chiếu.
7	Reproducibility và Versioning	4/4	PostgreSQL reproduction trả verdict MATCHED.
8	Market Data và Candlestick	4/4	Có Binance adapter và controlled/ browser evidence; LIVE timestamp cần xuất hiện trong video.
9	Strategy Engine ≥ 4 strategy	4/4	Bốn production plugin đã được kiểm chứng.
10	Composite Strategy	4/4	Có Majority Vote và Weighted Vote trong composite library; Search demo sử dụng Majority Vote.

STT	Tiêu chí	Tự đánh giá	Ghi chú evidence
11	Backtesting Engine	4/4	Có deterministic code/test; LIVE Result ID cần đối chiếu trong video.
12	Strategy Evaluation	4/4	Có đủ Return, Win Rate, MDD và Number of Trades.
13	Search và Stop Condition	4/4	Có deterministic Random Search, finite stop condition và bounded refill.
14	Leaderboard Top-K	4/4	Có deterministic projection; LIVE Revision ID cần xuất hiện trong video.
15	Visualization Strategy và Trade	4/4	Có UI implementation và controlled test; không dùng prototype làm LIVE evidence.
16	News Collector	4/4	Có provider/storage pipeline; LIVE source và timestamp cần đối chiếu.

STT	Tiêu chí	Tự đánh giá	Ghi chú evidence
17	Sentiment Analysis	4/4	Có versioned inference runtime; chưa có chiến lược sentiment ở production.
18	Source code và README	4/4	Repository, setup và run documentation đầy đủ.
19	Architecture Document	4/4	Có C4, module, deployment, data và dynamic views.
20	Architectural Decision Records	4/4	Có 17 ADR trong index tại thời điểm lập báo cáo.
21	Video/demo và link minh chứng	4/4	Link đã khai trong sheet; cần kiểm tra quyền truy cập và nội dung video.
22	Demo end-to-end	4/4	Đây là mức nhóm tự khai; evidence status chưa được nâng cho đến khi authenticated LIVE video được đối chiếu.
23	Scenario kiến trúc/xử lý lỗi	4/4	Có controlled scenario cho extension, recovery, isolation và reproduction.

STT	Tiêu chí	Tự đánh giá	Ghi chú evidence
24	Phần nâng cao có mục tiêu kiến trúc rõ	4/4	Mức tự khai; việc công nhận điểm cộng do giảng viên quyết định.

D.2 Đóng góp thành viên

Bảng D.2. Thông tin đóng góp đã có trong bảng tự đánh giá

STT	MSSV	Họ và tên	Đóng góp
1	23127221	Nguyễn Tiến Luật	25%
2	23127228	Phạm Văn Minh	25%
3	23127281	Đặng Nghi Văn	25%
4	23127494	Nguyễn Huỳnh Tiến	25%
			Tổng 100%

Chương E

Liên kết bàn giao và hướng dẫn tái lập

E.1 Repository

<https://github.com/PhamVanMinhBinhThuan/crypto-strategy-platform>

Repository là nguồn chính thức của source code, migration, automated tests, architecture documentation, ADR và evidence. Khi trích evidence cần ghi commit SHA tương ứng thay vì chỉ dùng link nhánh có thể thay đổi.

E.2 Video demo và báo cáo nộp

Video demo: <https://drive.google.com/drive/folders/12Tg8WE7-FiN0muv-9RtSavVMnm87NUb0?usp=sharing>

Hai liên kết cần được kiểm tra bằng cửa sổ ẩn danh trước khi nộp để bảo đảm giảng viên có thể truy cập mà không cần xin quyền.

E.3 Hướng dẫn setup, cài đặt và build

GitHub — docs/SETUP.md

Hướng dẫn bao gồm prerequisites, environment variables, PostgreSQL/Supabase migration, Redis, Sentiment, API, Worker, Web, build, test, health check và troubleshooting cho Windows/macOS/Linux.

E.4 Tài liệu kiến trúc liên quan

- Architecture index: <https://github.com/PhamVanMinhBinhThuan/crypto-strategy-platform/tree/main/docs/architecture>

- ADR index: <https://github.com/PhamVanMinhBinhThuan/crypto-strategy-platform/tree/main/docs/adr>
- API documentation: <https://github.com/PhamVanMinhBinhThuan/crypto-strategy-platform/tree/main/docs/api>
- Database documentation: <https://github.com/PhamVanMinhBinhThuan/crypto-strategy-platform/tree/main/docs/database>
- Demo runbook: <https://github.com/PhamVanMinhBinhThuan/crypto-strategy-platform/tree/main/docs/demo>